

Report IA

Corso di Intelligenza Artificiale

Federico Silvi Alessio Cicione Nicola Iadanza

Università degli Studi di Cassino e del Lazio Meridionale

LM 32 - Ingegneria Informatica

A.A. 2025–2026

Abstract

Il seguente report illustra l'implementazione e i risultati ottenuti circa le tesine del corso di Intelligenza Artificiale (LM-32) tenutosi presso l'Università degli Studi di Cassino e del Lazio Meridionale.

Ogni sezione dedicata alla singola tesina si sviluppa come segue:

- **Introduzione:** presentazione del problema proposto, degli obiettivi da raggiungere e dei vincoli da rispettare
- **Implementazione:** scelte implementative effettuate allo scopo di soddisfare le richieste della traccia
- **Risultati:** presentazione e analisi dei risultati ottenuti

Il codice implementativo può essere consultato nella seguente [repo](#).

L'appendice è formata da due sezioni:

- A Immagini: contiene le immagini troppo grandi per il formato a doppia colonna. Nel report vi sono vari elementi cliccabili per visionarle e, sopra ogni immagine, vi è un elemento cliccabile aggiuntivo per tornare nel punto in cui si stava leggendo
- B Codice: contiene del codice preso dal materiale caricato sul GitHub. Serve per seguire meglio il report nella sezione *Implementazione*. Similmente alle immagini vi sono elementi cliccabili

1 Tesina 1

1.1 Introduzione

Nella tesina 1 viene presentato un problema reale:

Una startup di telecomunicazioni deve progettare un'infrastruttura di rete wireless per connettere 20 siti strategici in una regione urbana/industriale, minimizzando i costi di installazione e massimizzando l'efficienza della connessione

Le specifiche del problema sono le seguenti:

- Rete composta da 20 nodi di comunicazione
- Ogni nodo rappresenta un sito potenziale per stazioni ripetitori
- I costi di connessione variano in base a

- Distanza geografica
- Caratteristiche (latenza, vulnerabilità, consumo energetico)
- Investimento infrastrutturale

Gli obiettivi da raggiungere sono:

- Implementare l'algoritmo per ottenere il Minimum Spanning Tree (MST) tramite Kruskal e Prim
- Ottimizzare la connessione minimizzando:
 - Costi di installazione
 - Lunghezza totale dei cavi/conessioni
 - Impatto sulle performance

Rispettando i seguenti vincoli:

- Tutti i 20 nodi devono essere connessi
- Considerare le caratteristiche specifiche di ogni scenario
- Gestire differenti tipologie di costi di connessione

1.1.1 Fasi del Progetto

Sviluppo dell'algoritmo

Implementare MST con criteri personalizzati.

Considerare:

- Costi di connessione
- Qualità del segnale
- Caratteristiche dello scenario

Simulazione degli Scenari

- Scenario Smart City IoT:

Obiettivo	Connettere sensori IoT in ambiente urbano
Max latenza per hop	50 ms
Fattore banda	1.5
Vincoli	Minimizzare latenza, considerare requisiti di banda

Tipi di nodi:

- Sensori traffico (alta priorità, colore rosso)
- Sensori ambientali (media priorità, colore arancione)
- Luci smart (bassa priorità, colore blu)

- Scenario Zona Sismica:

Obiettivo	Garantire ridondanza di comunicazione
Fattore ridondanza	2.5
Max vulnerabilità	0.65
Vincoli	Massimizzare affidabilità connessioni

Soglie vulnerabilità: Alta (>0.85), Media (0.55), Base (0.35)

- Scenario Ottimizzazione Energetica:

Obiettivo	Minimizzare consumo energetico
Max potenza per nodo	85 W
Budget totale potenza	1350 W
Vincoli	Limitare numero ripetitori

1.1.2 Struttura della Rete

Struttura della rete:

Area	Nodi	Priorità
Centro	0,1,2,3	Media priorità
Nord	4,5,6,7	Alta priorità
Sud	8,9,10,11	Bassa priorità
Est	12,13,14,15	Media priorità
Ovest	16,17,18,19	Media priorità

(Figura [Rete Iniziale](#))

1.2 Implementazione

1.2.1 Algoritmo MST

Per la creazione degli MST, si sono implementati due tipi di algoritmi:

- Kruskal (contenuto in [kruskal.py](#))
- Prim (contenuto in [prim.py](#))

1.2.2 Scenari

Funzioni di Costo

Affinché gli MST, ottenuti sia tramite Kruskal che Prim, rispettassero obiettivi e vincoli dettati dallo scenario, si sono dovute implementare diverse funzioni di costo per favorire i collegamenti più idonei (contenute in [cost.py](#)).

Distinguendo i vari scenari, abbiamo seguito il seguente iter:

- Smart City IoT ([codice](#)) :
 - Calcolo la distanza

- Calcolo della latenza e penalizzazione in caso di sfioramento del vincolo
- Calcolo della priorità sulla base della difficoltà del terreno
- Calcolo dell'efficienza di banda e penalizzazione in caso di sfioramento del vincolo
- Il costo si compone come il prodotto della somma del costo di latenza e di banda con la priorità

- Zona Sismica ([codice](#)):

- Calcolo vulnerabilità e penalizzazione in caso di sfioramento del vincolo
- Calcolo del fattore di stabilità come il rapporto tra la potenza media a disposizione dei due nodi e la potenza richiesta dal collegamento
- Calcolo della penalizzazione di stabilità come il reciproco del fattore di stabilità
- Il costo si compone come il prodotto della somma della vulnerabilità e della penalizzazione di stabilità con il fattore di ridondanza

- Ottimizzazione Energetica ([codice](#)) :

- Calcolo della potenza richiesta
- Calcolo della potenza richiesta per nodo
- Penalizzazione in caso si superi la capacità del nodo
- Calcolo del carico sul nodo come il rapporto tra la potenza richiesta per nodo e l'effettiva capacità dello stesso, con penalizzazione in caso di sfioramento del vincolo
- Il costo si compone come la somma tra la potenza richiesta e il carico

Funzioni di risoluzione

Le funzioni di risoluzione dei vari scenari sfruttano le funzioni appena descritte per calcolare il costo di ogni arco prima che la rete venga usata come ingresso per gli algoritmi di MST.

In particolare, per tutti gli scenari, ci si basa sul seguente pseudo-codice:

```

1: cost ← cost_function
2: apply_new_cost_to_network(cost)
3: if algorithm == 'kruskal' then
4:   mst_solver ← KruskalMST(network)
5: else
6:   mst_solver ← PrimMST(network)
7: end if
8: mst_edges ← mst_solver.find_mst()
9: passed, error_edges ← validate_scenario()
10: return mst_edges, passed, error_edges

```

dove

- `mst_edges`: archi dell'MST trovato dall'algoritmo
- `passed`: indica se la soluzione trovata rispetta i vincoli dello scenario
- `error_edges`: collegamenti che eventualmente sfiorano i vincoli

Main

All'interno del `main.py` viene definito un logging personalizzato in modo da informare l'utente circa lo svolgersi delle operazioni.

Lo stesso utente può avviare il programma da terminale tramite il seguente comando

```
python main.py -scenario scenario -algorithm alg
```

dove

- `scenario`:
 - `smartcity`
 - `seismic`
 - `energy`
- `alg`
 - `kruskal`
 - `prim`

A seconda dello scenario e dell'algoritmo scelti dall'utente, il programma verrà eseguito di conseguenza e i risultati saranno presentati sia graficamente che a terminale.

1.3 Risultati

Applicando gli algoritmi (risultati tra Prim e Kruskal identici) e le funzioni di costo descritti nei vari scenari proposti, si ottengono i seguenti risultati:

- Smart City IoT: l'MST trovato rispetta i vincoli imposti dalla traccia. Per visualizzare la rete risultante fare riferimento a [Rete Smart City](#), mentre il logging a terminale può essere consultato in [Terminal Smart City](#)
- Zona Sismica: l'MST trovato non rispetta i vincoli imposti dalla traccia a causa della presenza dei nodi 4, 5, 6, 7, i quali creano un sottografo le cui connessioni avranno tutte vulnerabilità superiori rispetto al vincolo imposto. Pertanto si è deciso di restituire lo stesso l'MST trovato ([Rete Zona Sismica](#)), notificando però all'utente i collegamenti che violano i vincoli ([Terminal Zona Sismica](#)).
- Ottimizzazione Energetica: l'MST trovato rispetta i vincoli imposti dalla traccia. Per visualizzare la rete risultante fare riferimento a [Rete Ottimizzazione Energetica](#), mentre il logging a terminale può essere consultato in [Terminal Ottimizzazione Energetica](#)

Si osservi come le soluzioni trovate negli scenari Smart City IoT e Ottimizzazione Energetica coincidono. Ciò è imputabile al fatto che le funzioni di costo ideate per entrambi gli scenari dipendano per gran parte dalla distanza tra i nodi.

2 Tesina 2

2.1 Introduzione

Nella tesina 2 viene presentato un problema reale:

Un team di ricerca deve analizzare l'impatto dei diversi parametri dell'algoritmo Particle Swarm Optimization sulla loro performance e convergenza utilizzando il dataset DARWIN. Il dataset contiene caratteristiche di scrittura a mano per lo studio dell'Alzheimer, offrendo un contesto reale e significativo per l'analisi dell'algoritmo PSO

Le specifiche del dataset DARWIN sono le seguenti:

- Feature di scrittura a mano
- Caratteristiche cinematiche
- Pressione della penna
- Parametri geometrici
- Caratteristiche temporali

e ha un'alta complessità a causa di:

- Categorie di feature multiple
- Dati numerici continui
- Correlazioni complesse

Gli obiettivi da raggiungere sono i seguenti:

- Implementare un Algoritmo PSO base per feature selection
- Studiare sistematicamente l'effetto dei parametri
- Determinare configurazioni ottimali

Rispettando i seguenti vincoli:

- Utilizzo stesso seed per confronti equi
- Minimo 30 run per configurazione
- Gestione missing values

2.1.1 Fasi del Progetto

Implementazione PSO base

Sviluppare algoritmo PSO con:

- Rappresentazione binaria per feature selection
- Funzione fitness basata su correlation analysis
- Aggiornamento velocità e posizione

Implementare logging dettagliato

- Fitness per iterazione
- Feature selezionate
- Tempi di esecuzione
- Diversità dello sciame

Analisi Parametrica

- Scenario Dimensioni Sciame:

Dimensione Sciame	20	50	100	200	500
-------------------	----	----	-----	-----	-----

Parametri fissi:

- Inerzia (w): 0.7
- Coefficiente cognitivo (c_1): 2.0
- Coefficiente sociale (c_2): 2.0

- Scenario Coefficienti PSO:

- Parametri variabili:

Inerzia (w)	0.4	0.6	0.7	0.9
Coefficiente cognitivo (c_1)	1.0	1.5	2.0	2.5
Coefficiente sociale (c_2)	1.0	1.5	2.0	2.5

- Dimensione sciame fissa: 100

- Scenario Criteri di Stop:

- Parametri variabili:

Numero iterazioni	50	100	200
Soglie	10	20	30
Tolleranze	1e-4	1e-5	1e-6

2.2 Implementazione

2.2.1 PSO

L'algoritmo di PSO ([pso_feature_selection.py](#), classe `ParticleSwarmOptimization`) è stato implementato con le seguenti funzioni:

- `__init__`: inizializza i parametri del PSO
- `_initialize_swarm`: inizializza tutte le particelle del PSO (fare riferimento alla classe `Particle`)
- `evaluate_swarm`: valuta il fitness di tutte le particelle dello sciame. Per ogni particella viene calcolata la qualità della soluzione (sottoinsieme di feature selezionate) tramite una funzione di fitness basata sulla correlazione (definita in `fitness_correlation_based`) con la variabile target
- `update_gbest`: aggiorna il miglior valore di fitness globale trovato
- `run`: esegue l'algoritmo cardine del PSO tramite i seguenti step:
 1. Valutazione dello swarm ed eventuale aggiornamento del global best
 2. Valutazione di interruzione prematura dell'algoritmo: si controlla se il miglioramento del fitness globale è inferiore rispetto alla soglia di tolleranza; quindi se si supera il numero massimo di iterazioni senza miglioramento degno di nota, l'algoritmo viene interrotto

3. Aggiornamento di velocità (fare riferimento alla funzione `update_velocity`) e della posizione (fare riferimento alla funzione `update_position`) di ogni particella

- `calculate_swarm_dispersion`: calcola la dispersione dello sciame, la quale misura il grado di diversità genetica o strutturale tra le particelle. Viene calcolata come la media delle distanze di Hamming tra tutte le possibili coppie di particelle nello sciame
- `calculate_average_velocity`: calcola la velocità media dello sciame

dove sono fondamentali le funzioni:

- `update_velocity`: Aggiornamento della velocità della particella seguendo la formula

$$v = w \cdot v_{part} + c_1 \cdot r_1 \cdot (P_{best} - pos) + c_2 \cdot r_2 \cdot (G_{best} - pos)$$

dove

- w : inerzia
- v_{part} : velocità attuale della particella
- c_1 : coefficiente cognitivo
- r_1, r_2 : coefficiente randomico tra 0 e 1
- P_{best} : migliore posizione della particella considerata
- pos : posizione attuale della particella
- c_2 : coefficiente sociale
- G_{best} : migliore posizione globale
- `update_position`: A partire dalla velocità della particella, questa viene convertita in probabilità dalla $V - Shaped Function$ (riferimento a `v_shaped_prob`), scelta al posto della *sigmoid* poiché non richiede pre-processing sulla velocità stessa.

Quindi si generano dei valori casuali e si comparano con le probabilità date dalle componenti della velocità: se la probabilità è più alta del valore randomico si ha un flip del bit nella posizione corrispondente, altrimenti si mantiene quello presente

2.2.2 Logger

Il logger per gli esperimenti (fare riferimento alla classe `ExperimentLogger`, prevede le seguenti funzioni:

- `__init__`: inizializzazione delle variabili interne
- `log_iteration`: salva all'interno di un dizionario le informazioni rilevanti relative ad un'iterazione
- `log_run`: salva all'interno di un dizionario le informazioni rilevanti relative ad una run

2.2.3 Scenari

Per implementare gli scenari richiesti si hanno le seguenti 3 funzioni:

- `run_experiment_swarm_size`: Implementa lo scenario 1, in cui bisogna variare le dimensioni dello

sciame, lasciando invariati i parametri del PSO ($w = 0.7, c_1 = 2.0, c_2 = 2.0$).

Quindi per ogni dimensione dello sciame, viene chiamata un'istanza della classe `ParticleSwarmOptimization`, viene chiamato il suo metodo `run` e i risultati vengono opportunamente salvati all'interno di un file `.txt` e in un dizionario di logger, le cui chiavi sono le dimensioni dello sciame stesse

- `run_experiment_pso_coefficients`: Implementa lo scenario 2, in cui bisogna variare i coefficienti del PSO (w, c_1, c_2), lasciando invariata la dimensione dello sciame (100).

Quindi per ogni combinazione dei coefficienti, viene chiamata un'istanza della classe `ParticleSwarmOptimization`, viene chiamato il suo metodo `run` e i risultati vengono opportunamente salvati all'interno di un file `.txt` e in un dizionario di logger, le cui chiavi sono le combinazioni dei coefficienti

- `run_experiment_stopping_criteria`: Implementa lo scenario 3, in cui bisogna variare il numero di iterazioni, le soglie di non-miglioramento e la tolleranza, lasciando invariati i coefficienti e le dimensioni dello sciame ($w = 0.7, c_1 = 2.0, c_2 = 2.0, dim = 100$).

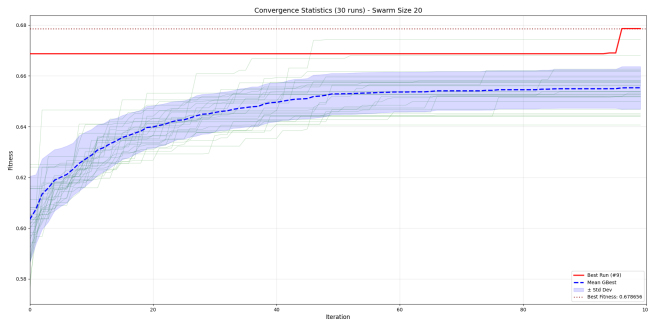
Quindi per ogni combinazione dei criteri di stop, viene chiamata un'istanza della classe `ParticleSwarmOptimization`, viene chiamato il suo metodo `run` e i risultati vengono opportunamente salvati all'interno di un file `.txt` e in un dizionario di logger, le cui chiavi sono le combinazioni dei criteri di stop

2.3 Risultati

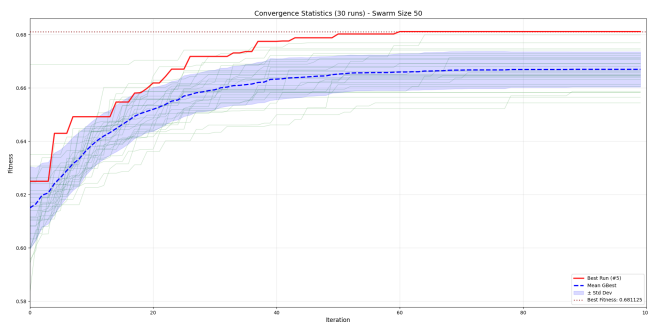
2.3.1 Scenario Dimensione Sciame

Curve di Convergenza

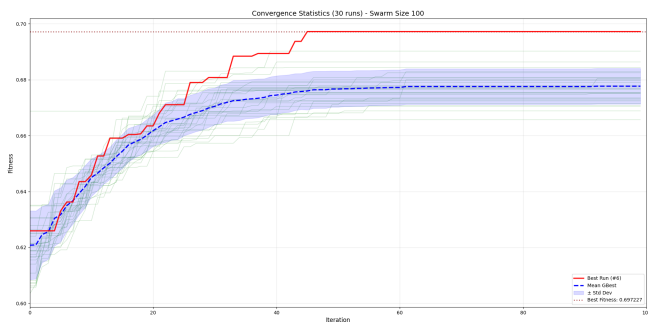
Facendo riferimento alle curve di convergenza con 20 particelle:



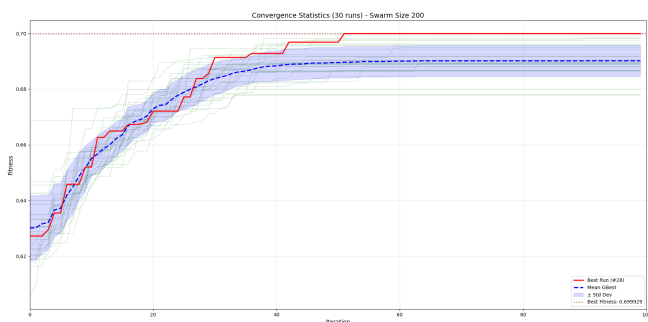
con 50 particelle:



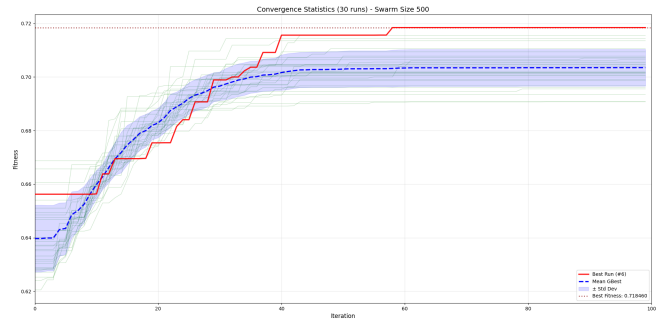
con 100 particelle:



con 200 particelle:



e con 500 particelle:



possiamo osservare:

- Il fitness medio attorno alla 50-sima iterazione
- Aumentando il numero di particelle si ha un miglioramento generale del miglior valore di fitness e il comportamento medio di ogni configurazione migliora, rendendosi via via più indipendente dall'inizializzazione randomica, poiché al crescere del numero di particelle si esplora di più all'interno dello spazio

La Figura [conv all](#) rappresenta sullo stesso grafico le migliori run per ogni dimensione dello sciame, assieme alla fitness media dello sciame ad ogni iterazione.

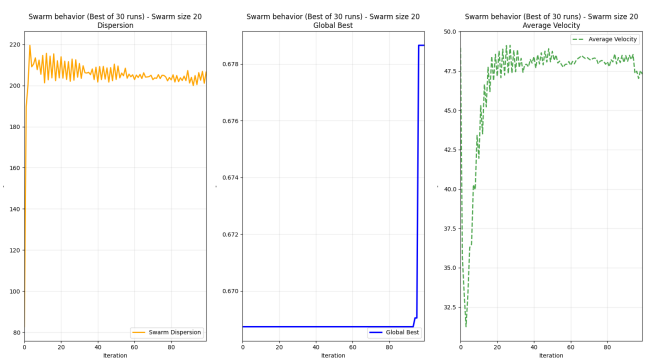
È da considerare che all'aumento del numero di particelle si ha una maggiore esplorazione con conseguente miglioramento del fitness medio, ma quest'ultimo diventa sempre più esiguo, sintomo del fatto che ci si sta avvicinando alla saturazione dello spazio. Come si vedrà in seguito, l'aumento della dimensione dello sciame aumenta in maniera importante il tempo medio richiesto.

Swarm Behaviour

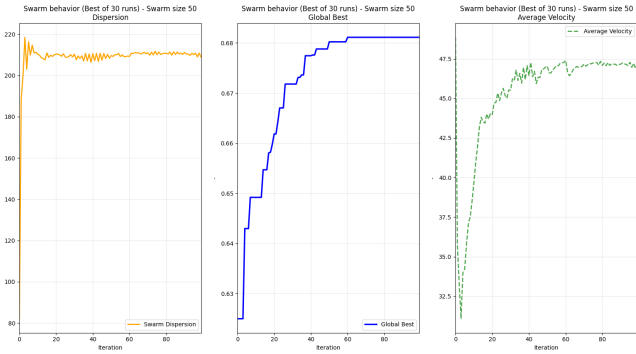
Prendendo in considerazioni grafici che rappresentano il comportamento delle grandezze:

- Dispersione
- Global Best
- Velocità media

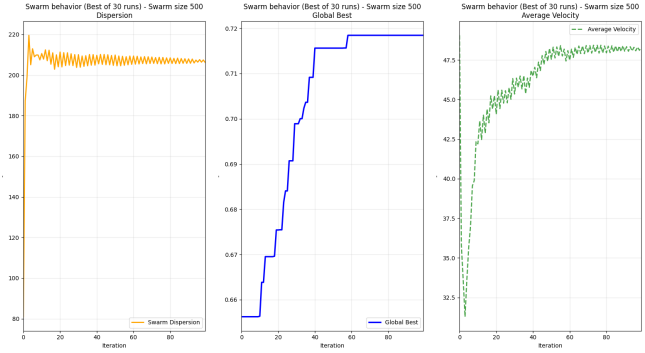
con 20 particelle:



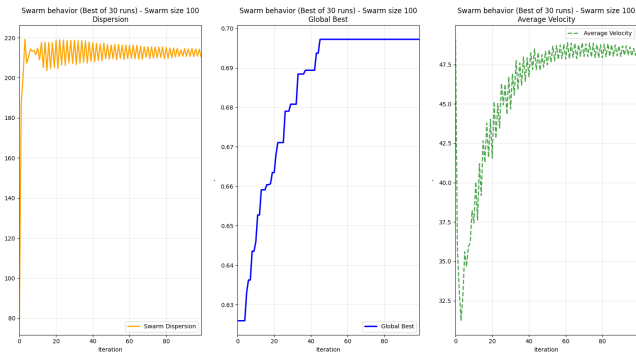
con 50 particelle:



e con 500 particelle:



con 100 particelle:

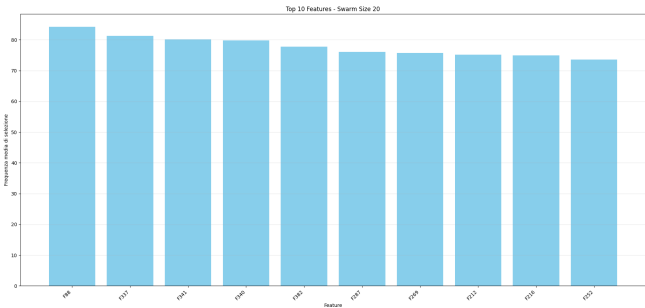


Possiamo osservare come, nel caso in esame, variare il numero di particelle non influenzi né la dispersione né la velocità media, mentre la seconda varia come già osservato.

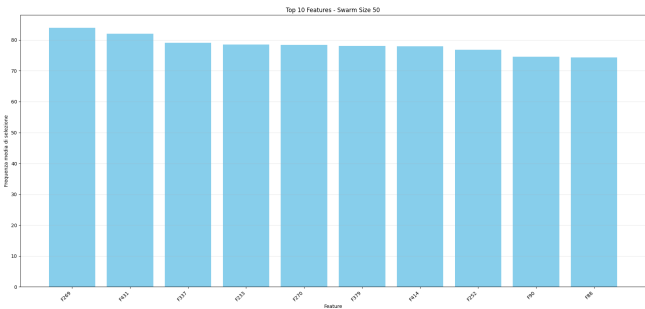
Feature

Le seguenti figure rappresentano le top 10 feature selezionate per ogni configurazione.

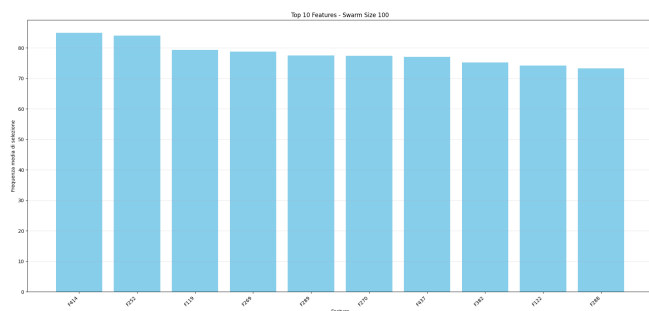
Con 20 particelle:



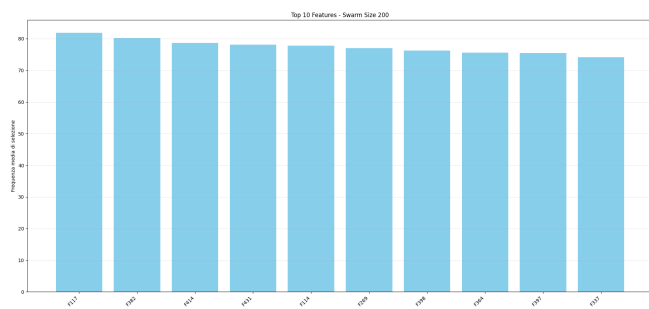
Con 50 particelle:



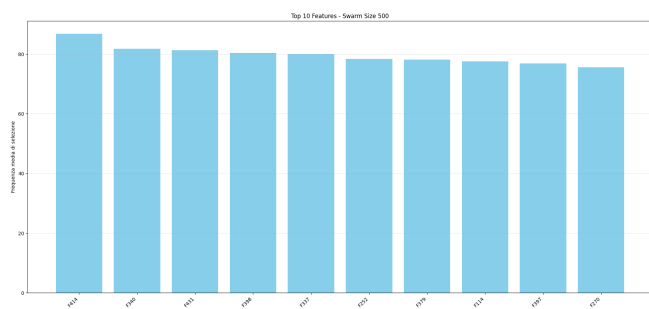
Con 100 particelle:



Con 200 particelle:



Con 500 particelle:

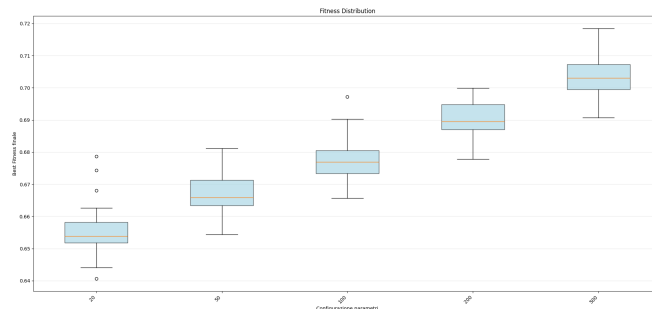


Possiamo osservare:

- In generale, un sottoinsieme di feature compare in quasi tutte le configurazioni (come ad esempio F337,F414,F252), mentre altre compaiono in maniera più sporadica. Questo è indice della forte correlazione tra le feature ricorrenti e il target. In particolare l'ordine di importanza del sottoinsieme ricorrente viene modificato all'aumentare del numero di particelle

Box Plot

La seguente figura rappresenta i box plot delle configurazioni trattate:

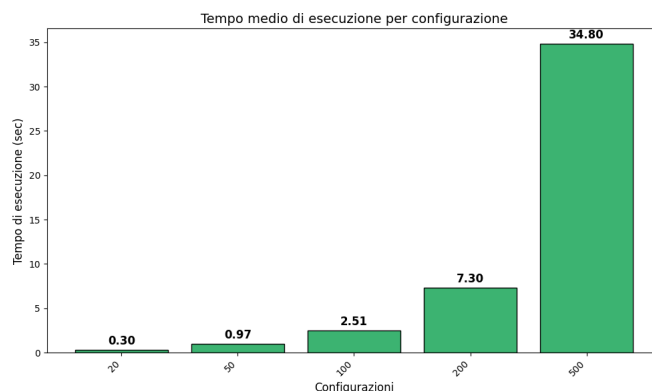


Possiamo osservare:

- Aumento della fitness al crescere del numero di particelle: ciò conferma quello che è stato osservato già in precedenza
- Diminuzione di outlier poiché si diventa più robusti alla randomizzazione dell'inizializzazione delle particelle

Tempi di Esecuzione

La seguente figura rappresenta i tempi di esecuzione delle varie configurazioni:



Si osserva un aumento drastico del tempo di esecuzione a partire dalle configurazioni con 100 particelle, poiché tutte le operazioni eseguite su una singola particella devono essere ripetute per l'intero sciame ad ogni iterazione.

2.3.2 Scenario Coefficienti PSO

Box Plot

La figura [Coeff. Box Plot](#) rappresenta i box plot del fitness di tutte le configurazioni sui coefficienti del PSO.

Possiamo osservare che:

- Le migliori configurazioni hanno tutte inerzia bassa
- Aumentando l'inerzia si ha un sensibile degrado del fitness finale

- Le configurazioni peggiori sono quelle con inerzia alta e con forte sbilanciamento tra coefficiente cognitivo e sociale

Ciò implica che:

- Il PSO beneficia di esplorazione contenuta
- Il PSO ha un degrado sui risultati a fronte di un'inerzia alta
- Il PSO ha un ulteriore degrado se si tende fortemente o verso il personal best della particella o verso il global best dello sciame

Tempi di Esecuzione

La figura [Coeff. Execution Time](#) rappresenta i tempi di esecuzione delle varie configurazioni.

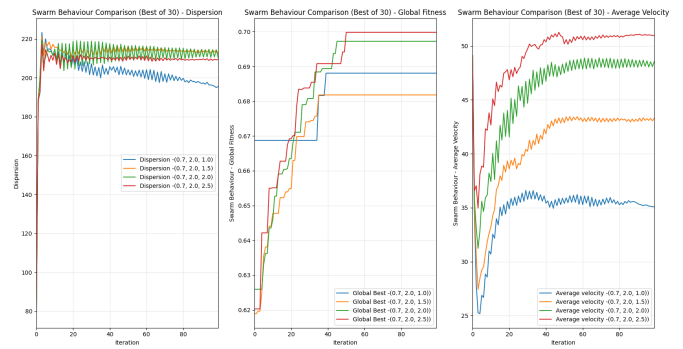
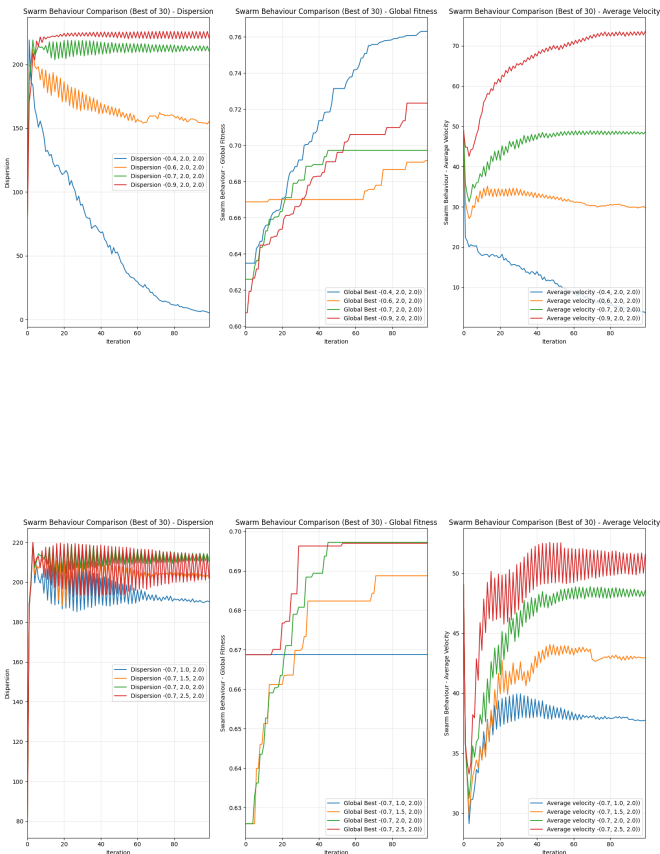
Possiamo osservare che essi sono equiparabili → variare i coefficienti del PSO non influisce molto sui tempi di esecuzione.

Swarm Behaviour

Le seguenti figure rappresentano:

- Dispersione
- Global Best
- Velocità media

a fronte delle variazioni dei singoli coefficienti rispetto alla configurazione di default ($w = 0.7, c_1 = 2.0, c_2 = 2.0$):



Possiamo osservare che:

- Al crescere dell'inerzia aumentano dispersione e velocità media a causa della maggiore esplorazione, mentre il fitness diminuisce
- Al crescere del coefficiente cognitivo:
 - Aumentano dispersione e velocità media, assieme alla loro oscillazione. Questo è imputabile al fatto che c'è una "lotta" costante tra coefficiente cognitivo e sociale
 - Aumenta il fitness a causa della diminuzione del divario tra coefficiente sociale e cognitivo, in accordo con l'osservazione già fatta
- Al crescere del coefficiente sociale:
 - Aumentano dispersione e velocità media, ma non assieme alla loro oscillazione. Questo è imputabile al fatto che il global best è comune a tutto lo sciame, mentre il personal best è proprio di ogni particella
 - Aumenta il fitness a causa della diminuzione del divario tra coefficiente sociale e cognitivo, in accordo con l'osservazione già fatta

Configurazione Migliore

La figura [Best Convergence Curve](#) mostra la curva di convergenza della configurazione migliore, mentre la figura [Best Feature Selection](#) mostra le feature selezionate.

Su quest'ultima possiamo osservare che solo 4 feature su 10 sono in comune con le feature selezionate nella configurazione migliore (dimensione sciame pari a 500) dello scenario precedente.

2.3.3 Scenario Criteri di Stop

Box Plot

La figura [Stop Criteria Box Plot](#) rappresenta i box plot del fitness delle configurazioni dello scenario.

Possiamo osservare che

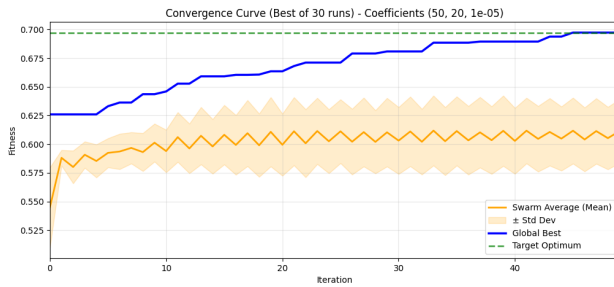
- Al crescere del numero di iterazioni massime si ha un aumento del fitness, che si stabilizza da 100 in su
- Con il numero di soglia pari a 10 si ha un leggero peggioramento del fitness rispetto agli altri casi, nei quali non solo si ha un miglioramento ma anche la scomparsa degli outlier

- Il variare della tolleranza non influenza il fitness finale
- Si raggiunge lo stesso risultato della configurazione di default

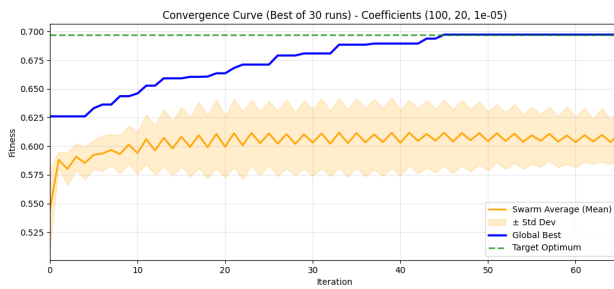
Curve di Convergenza

Le seguenti figure rappresentano le curve di convergenza per 3 run selezionate per ognuno dei valori del numero di iterazioni massime.

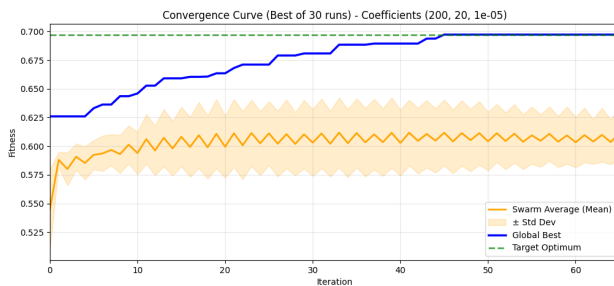
Con configurazione [$iter = 50$, $soglia = 20$, $tolleranza = 1e - 05$]:



con configurazione [$iter = 100$, $soglia = 20$, $tolleranza = 1e - 05$]



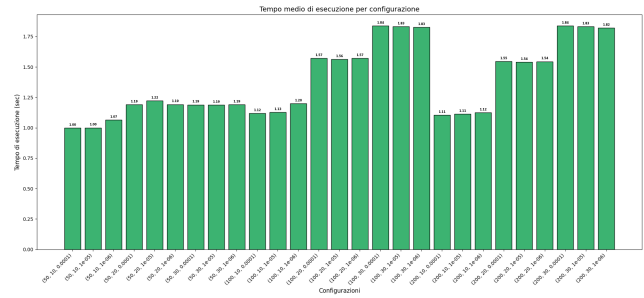
e con configurazione [$iter = 200$, $soglia = 20$, $tolleranza = 1e - 05$]



Possiamo osservare che: qualunque sia il numero di iterazioni massime, si ha l'interruzione dell'algoritmo tra la 40-sima e la 50-sima iterazione.

Tempi di Esecuzione

La seguente figura rappresenta i tempi di esecuzione delle configurazioni dello scenario:



Possiamo osservare che i tempi di esecuzione variano tra l'essere la metà e i $3/4$ del tempo di esecuzione dello scenario di default.

3 Tesina 3

3.1 Introduzione

Nella tesina 3 viene proposto un problema reale:

Un team di ricerca medica deve analizzare l'impatto dei diversi parametri delle Reti Neurali sulla loro performance di classificazione per la diagnosi dell'Alzheimer attraverso l'analisi della scrittura. Il dataset DARWIN contiene caratteristiche estratte da dati di scrittura a mano di 174 partecipanti, offrendo un contesto reale e significativo per l'ottimizzazione delle reti neurali nella diagnosi precoce delle malattie neurodegenerative

Le specifiche del dataset DARWIN sono le seguenti:

- Feature estratte da dati di scrittura a mano
- Classificazione binaria (paziente/controllo sano)
- 451 feature
- 174 istanze
- Dati numerici continui
- Alta dimensionalità
- Correlazioni complesse

Gli obiettivi da raggiungere sono:

- Implementazione di una rete neurale base per la classificazione usando scikit-learn
- Studiare sistematicamente l'effetto dei parametri
- Determinare configurazione ottimali

Rispettando i seguenti vincoli:

- Utilizzo dello stesso seed per confronti equi
- Minimo 30 run per configurazione
- Standardizzazione appropriata dei dati
- Cross-validation per ogni configurazione
- Gestione dell'overfitting

3.1.1 Fasi del Progetto

Implementazione Base

Sviluppare una rete neurale con:

- Configurazione base di `MLPClassifier`
- Preprocessing dei dati
- Train/test split appropriato con stratificazione

Implementare logging dettagliato con

- Accuracy per epoca
- Tempi di training
- Loss function
- Curve di apprendimento

Analisi Parametrica

Considerare vari scenari implementativi per analizzare l'effetto della variazione di specifici sottoinsiemi di parametri.

Scenario Architettura e Attivazione:

- Configurazioni Layer:

	Numero neuroni		
Singolo layer	200	400	600
Due layer	(400,200)	(600,300)	(800,400)
Tre layer	(400,200,100)	(600,300,150)	

- Funzioni di attivazione:

- 'identity'
- 'logistic'
- 'tanh'
- 'relu'

Scenario Learning Rate e Ottimizzazione:

L.r. init.	0.0001	0.001	0.01	0.1
L.r. pol.	constant	invscaling	adaptive	
Solver	adam	sgd	lbfgs	
Batch size	16	32	64	

dove

- L.r. init. = learning rate iniziali
- L.r. pol. = learning rate policy

Scenario Regularizzazione:

alpha	0.0001	0.001	0.1	0.1	0.5
Valid. split	0.1	0.15		0.2	
No change	5	10		20	
Early stop	True				

dove

- Valid. split = validation split

3.2 Implementazione

3.2.1 MLP base

Train/Test Split vs Cross Validation

L'addestramento dell'MLP prevede la seguente pipeline:

1. Caricamento dataset
2. Gestione missing values
3. Standardizzazione tramite `StandardScaler()`
4. Addestramento dell'MLP

In un primo momento sono state implementate due funzioni di addestramento: una nel caso di divisione del dataset tramite train/test (80%, 20%) split e l'altra per la cross validation (contenute in `train.py`).

Poiché la tesina 2 è incentrata su feature selection dello stesso dataset (DARWIN), si è pensato di addestrare l'MLP di default con Cross Validation in due scenari: senza e con feature selection (le feature risultanti dalla migliore configurazione della tesina 2 sono salvate nel file `best_feat_list`).

Ciò è stato fatto anche nell'ottica di evitare overfitting con reti neurali più complesse.

3.2.2 Logger

Le classi `MLPLogger` e `CVLogger` (fare riferimento a `logger.py`) implementano:

- Logger base per Train/Test Split
- Estensione del logger base per contenere anche le informazioni relative alla Cross Validation

3.2.3 Scenari

Tramite la funzione `get_mlp_config`, si estrapolano tutte le configurazioni a seconda dello scenario scelto.

Dopodiché si addestra ogni configurazione come già visto, salvando opportunamente i risultati all'interno di dizionari in cui le chiavi sono le configurazioni stesse.

3.3 Risultati

3.3.1 Scenario Base

In questa sezione vi sono i risultati ottenuti nello scenario di default.

Train/Test Split vs Cross Validation

Per quanto riguarda Train/Test Split vs Cross Validation, si sono osservati i seguenti risultati:

- Train/Test Split:

Metrica	Media	Std Dev
Test Accuracy	0.8305	± 0.0612
Sensitivity	0.8037	± 0.0857
Specificity	0.8588	± 0.0689
AUC	0.9148	± 0.0467

- Cross Validation:

Metrica	Media	Std Dev
Validation Accuracy	0.8368	± 0.0118
Sensitivity	0.8335	± 0.0163
Specificity	0.8404	± 0.0174
AUC	0.9177	± 0.0076

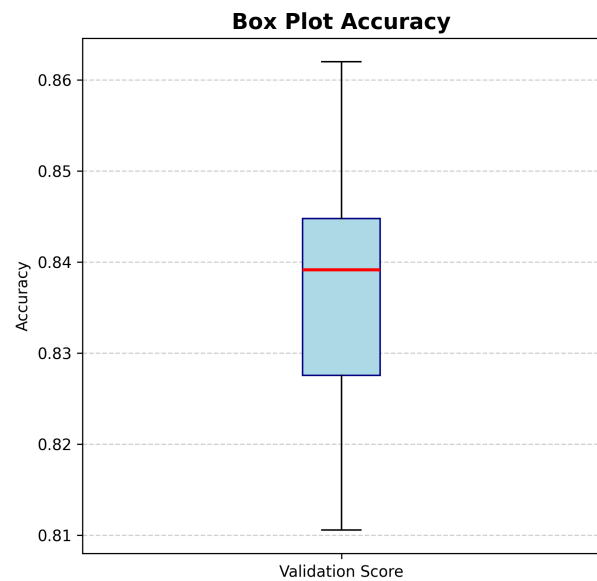
Com'era prevedibile le due modalità sono equiparabili per le medie delle metriche, ma la Cross Validation presenta una deviazione standard molto più bassa.

Feature Selection

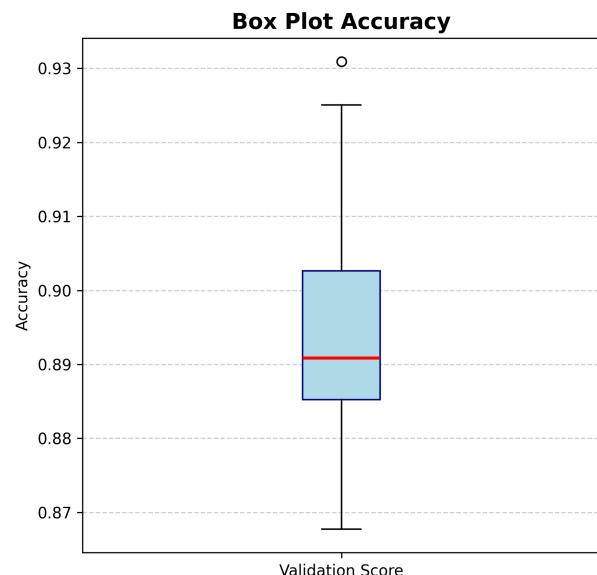
Nello scenario con e senza feature selection, graficando opportunamente le informazioni d'interesse (fare riferimento a `visualization.py`), si può fare un confronto:

- Accuracy:

Senza Feature Selection:

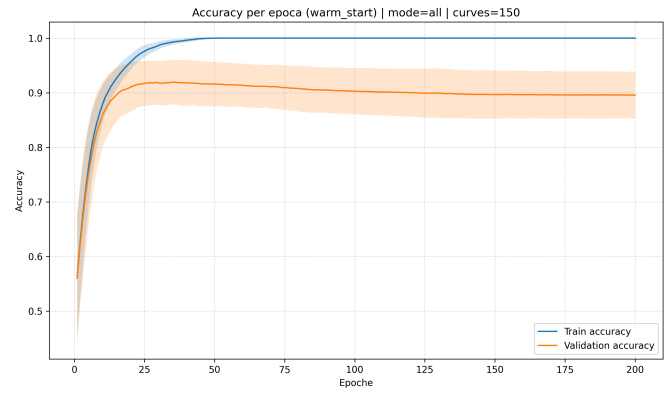
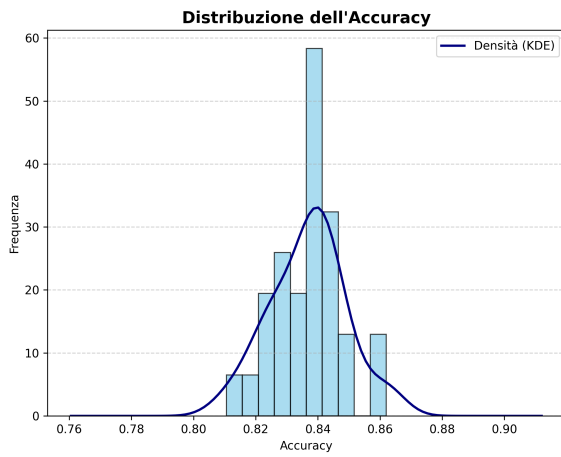


Con Feature Selection:



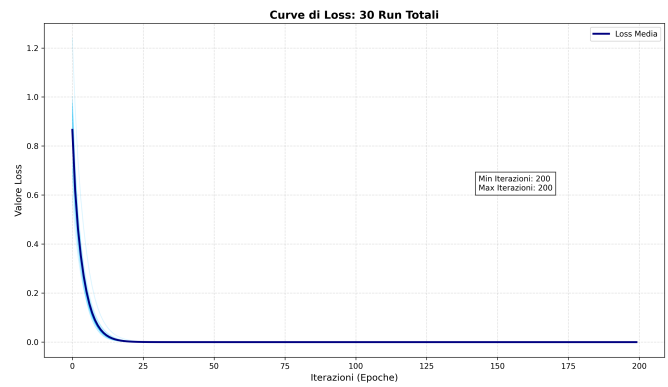
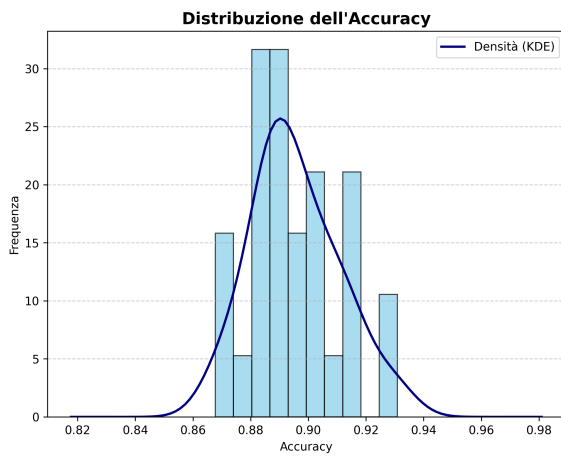
- Distribuzione Accuracy nella Cross Validation:

Senza Feature Selection:



- Curva di Loss:

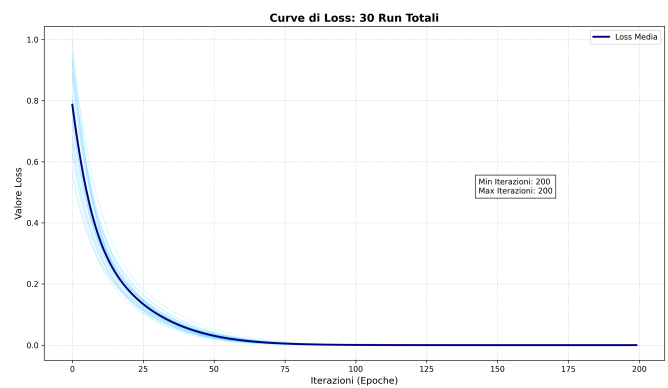
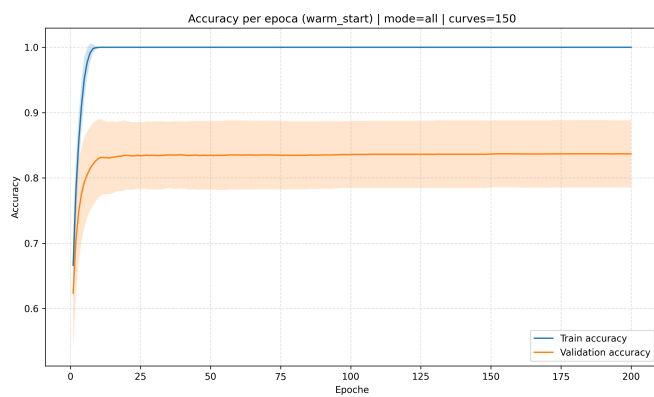
Con Feature Selection:



Con Feature Selection:

- Accuracy per Epoca:

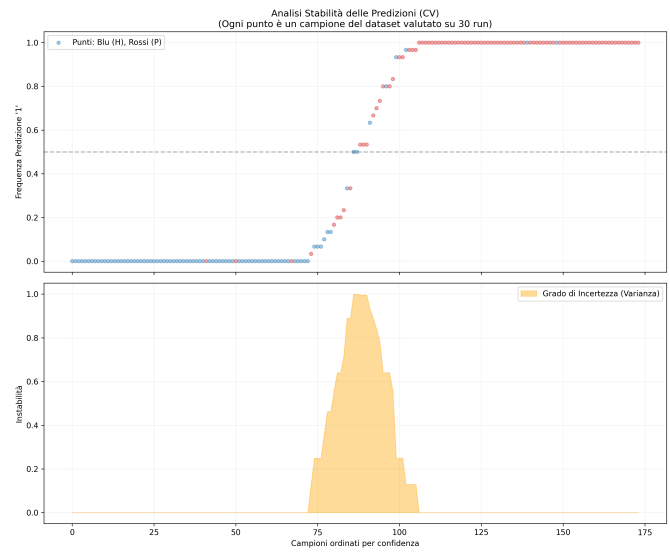
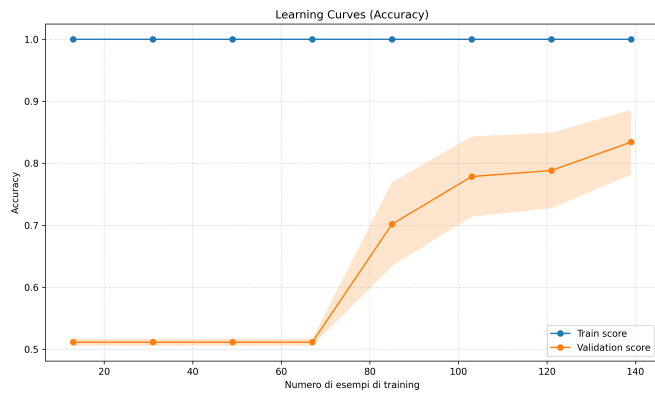
Senza Feature Selection:



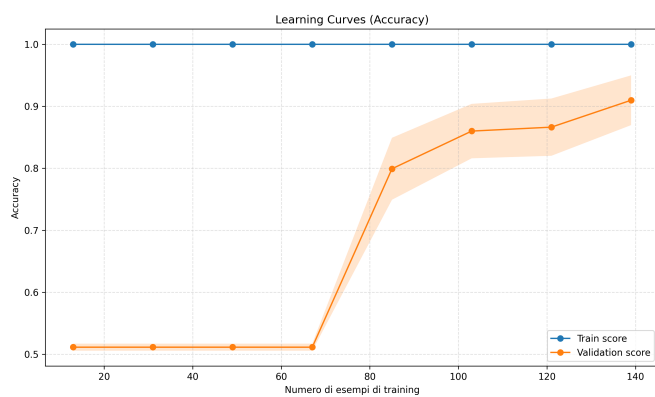
- Curva di apprendimento:

Con Feature Selection:

Senza Feature Selection:

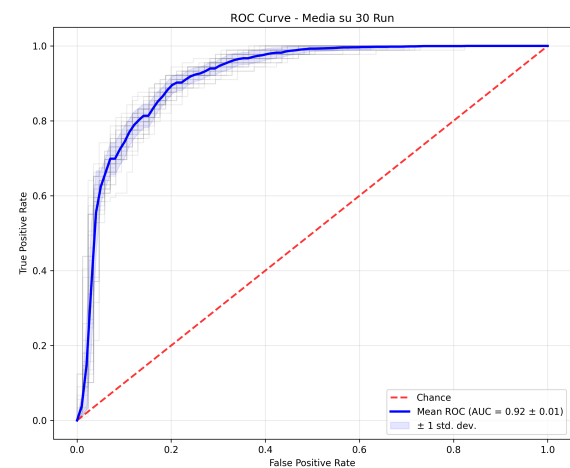


Con Feature Selection:



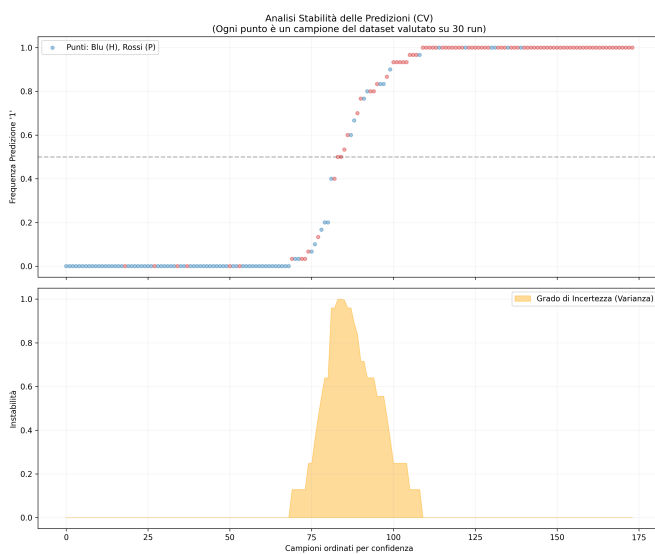
- Curva ROC:

Senza Feature Selection:

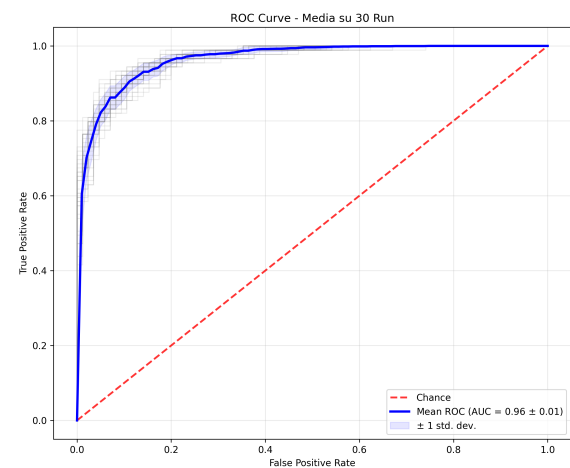


- Stabilità delle predizioni:

Senza Feature Selection:



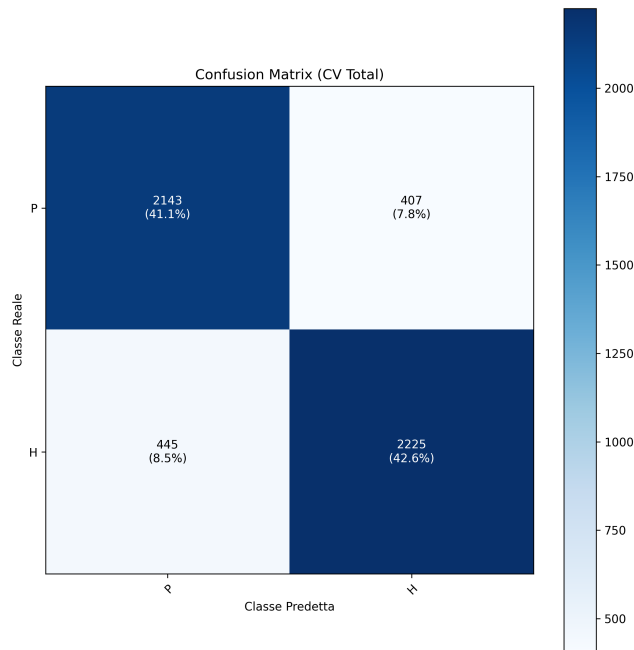
Con Feature Selection:



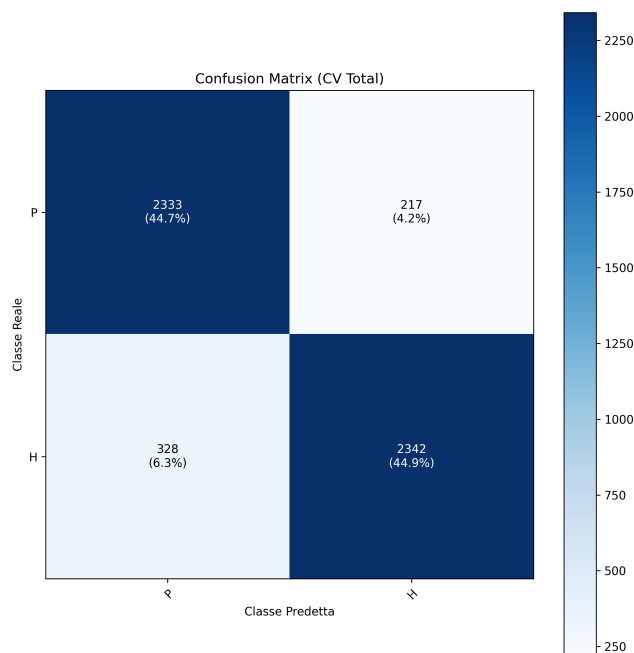
Con Feature Selection:

- Matrice di confusione

Senza Feature Selection:



Con Feature Selection:



Con feature selection si può osservare che:

- L'accuracy migliora sensibilmente
- La curva di loss converge più lentamente
- La stabilità delle predizioni aumenta
- Curva ROC più vicina al punto ideale
- Matrice di confusione migliore

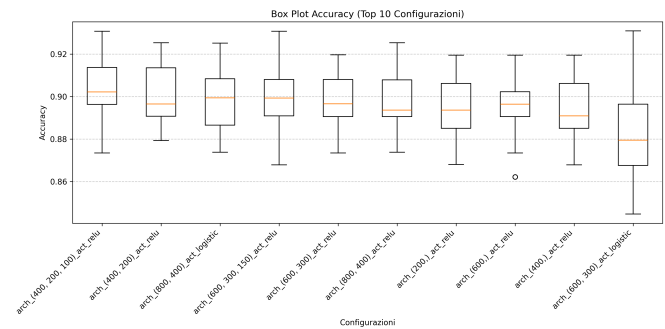
Per questo l'addestramento negli scenari che seguono è stato fatto con feature selection.

3.3.2 Scenario Architettura e Attivazione

Box Plot

La figura [Box Plot All](#) mostra le box plot di tutte le configurazioni.

Prendendo le top 10 configurazioni si ha:



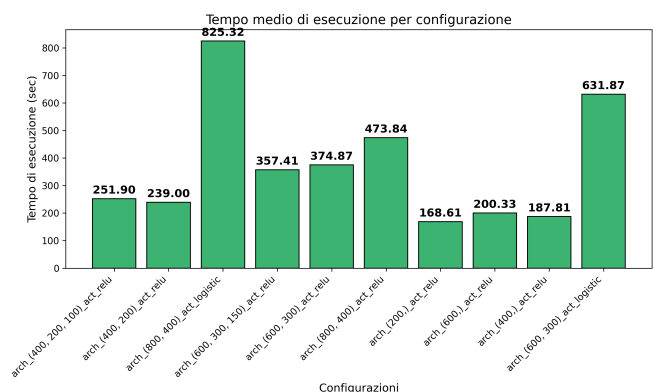
Si può osservare che:

- L'architettura (400, 200, 100) con attivazione *ReLU* mostra la mediana più alta (sopra lo 0.90)
- Le attivazioni *ReLU* sono preponderanti rispetto alle altre, con la sola eccezione di due attivazioni *logistic*
- Sebbene le configurazioni con *logistic* abbiano accuracy massima comparabile a quelle con *ReLU*, la loro mediana risulta abbastanza bassa e una in particolare presenta forte varianza

Tempi di Esecuzione

La figura [Execution Time All](#) mostra i tempi di esecuzione di tutte le configurazioni.

Prendendo le top 10 configurazioni si ha:

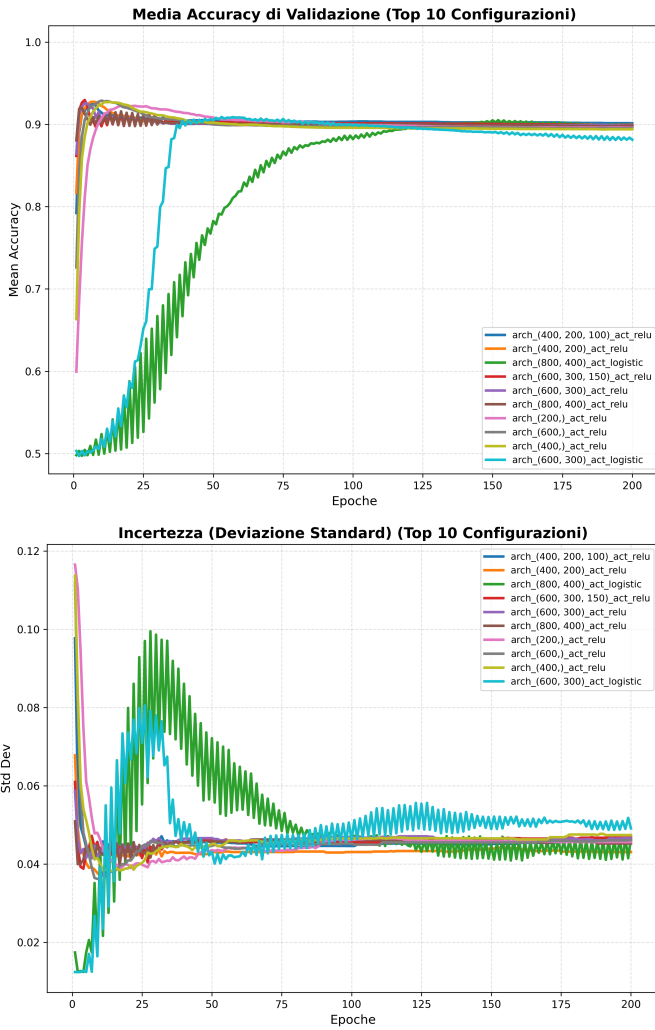


Si può osservare:

- All'aumentare del numero di neuroni aumenta anche il tempo di esecuzione
- L'attivazione *ReLU* presenta tempi molto più piccoli della *logistic* ed è quindi preferibile

Confronto Accuracy

La seguente immagine fa un confronto dell'accuracy media e della sua deviazione nelle top 10 configurazioni:

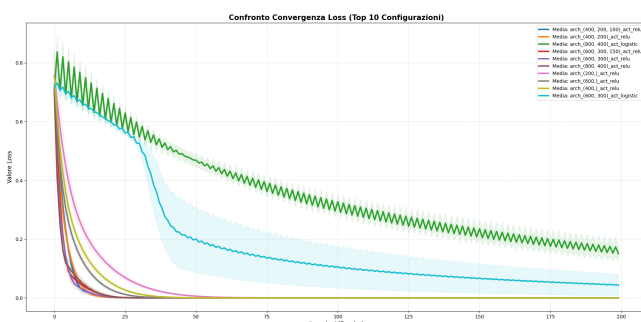


dove si può osservare:

- La maggior parte dei modelli *ReLU* converge molto rapidamente (entro le prime 25 epoche). Le configurazioni con funzione *logistic* (linea verde e azzurra) sono molto più lente, impiegando oltre 125 epoche per stabilizzarsi
- L'incertezza è altissima nelle prime fasi per *logistic*, mentre i modelli ReLU mostrano un picco di instabilità iniziale brevissimo per poi stabilizzarsi su valori di deviazione standard costanti e bassi

Curva di Loss

La prossima figura mostra le curve di loss per le top 10 configurazioni:

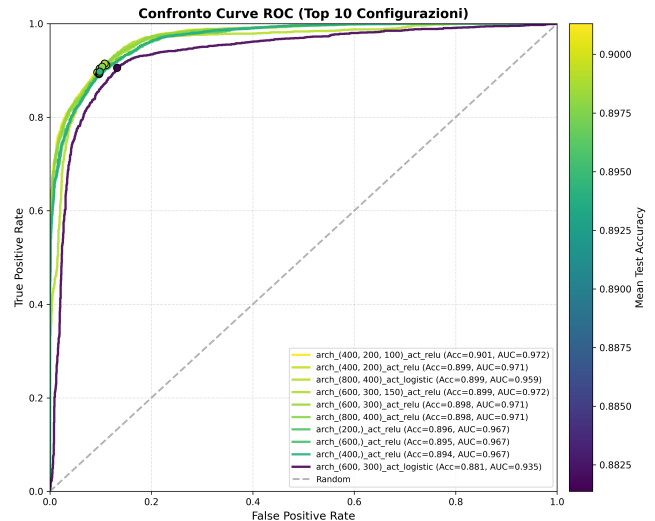


Si può notare:

- Attivazione *ReLU* molto più veloce in convergenza, mentre *logistic* è molto più lenta e presenta più oscillazione/deviazione

Curva ROC

Di seguito le curve ROC delle top 10 configurazioni:



dove si può osservare:

- Tutte le configurazioni sono molto vicine tra loro, tranne (600, 300) con *logistic*, che in generale presenta prestazioni inferiori

Stabilità Previsioni

La figura [Stability Comparison](#) mostra la stabilità delle previsioni: più il valore è vicino a 1, più la classe predetta è stata 1.

Si può osservare:

- Le configurazioni con *logistic* presentano un'instabilità relativa più alta: valori più bassi agli estremi e più alti nelle zone intermedie

3.3.3 Scenario Learning Rate e Ottimizzazione

Box Plot

Le figure [Box Plot 1](#), [Box Plot 2](#), [Box Plot 3](#) e [Box Plot 4](#) mostrano le box plot di tutte le configurazioni.

Possiamo osservare:

- Per learning rate iniziale = 0.0001:
 - Con Solver = Adam si ha l'accuracy migliore, che aumenta all'aumentare delle dimensioni del batch
 - Con Solver = SGD si ha l'accuracy peggiore sia per mediana che per varianza, le quali peggiorano con l'aumentare delle dimensioni del batch
 - Con Solver = lbfgs si ha una buona accuracy che non cambia con il variare delle dimensioni del batch

- Le accuracy risultano indipendenti dalla policy sul learning rate
- Per learning rate = 0.001:
 - Adam diventa il solver con accuracy peggiore (migliora all'aumentare delle dimensioni del batch)
 - Sgd diventa il solver migliore, aumentando l'accuracy in concordanza con le dimensioni del batch
 - Lbfgs rimane il solver di mezzo
 - Si ha sempre indipendenza dalla policy
- Per learning rate = 0.01 e 0.1 si nota il pattern dell'osservazione precedente con un leggero peggioramento dell'accuracy globale

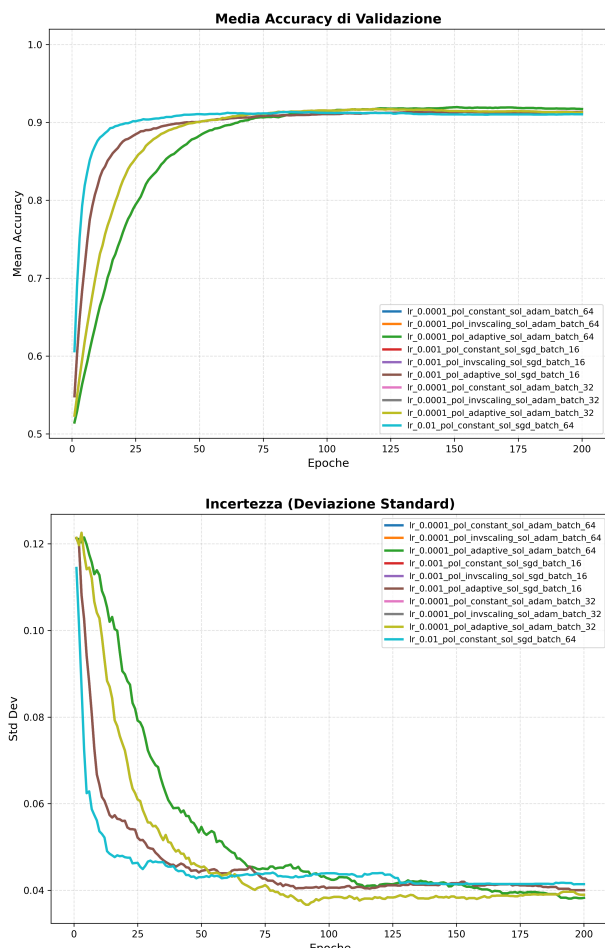
Tempi di Esecuzione

La figura [Execution Time 1](#), [Execution Time 2](#), [Execution Time 3](#), [Execution Time 4](#) mostra i tempi di esecuzione di tutte le configurazioni.

Si può osservare che i tempi di esecuzione sono comparabili tra loro e variano da 200 a 300 secondi circa.

Confronto Accuracy

La seguente immagine fa un confronto dell'accuracy media e della sua deviazione nelle top 10 configurazioni:

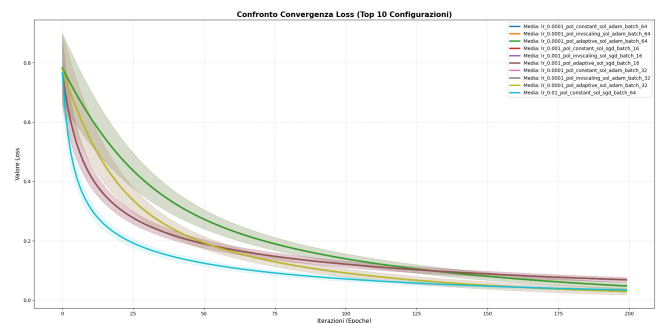


Si può notare:

- Le curve con lo stesso learning rate, solver e dimensione del batch ma policy diverse sono esattamente sovrapposte, a conferma dell'osservazione già fatta
- Le due curve verdi rappresentano la stessa famiglia di risultati ma con dimensioni di batch differenti: dimensione più piccola migliora di molto la velocità di convergenza, ma peggiora lievemente l'accuracy finale
- All'aumentare del learning rate si ha un miglioramento generale della convergenza a fronte di un lieve calo dell'accuracy finale. Il comportamento è speculare nel grafico dell'incertezza

Curva di Loss

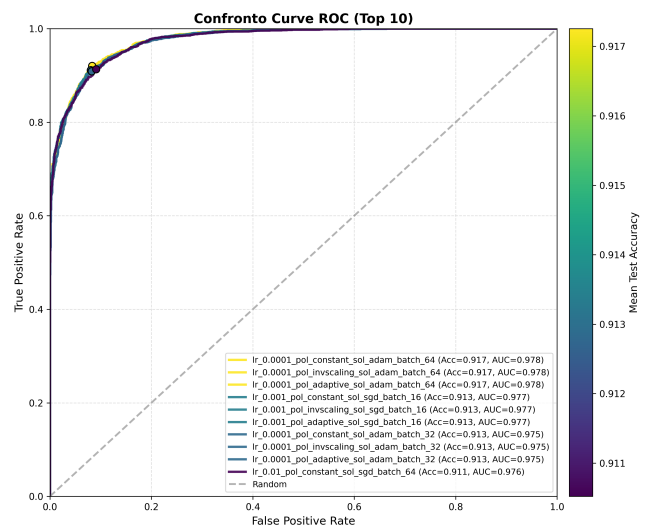
La prossima figura mostra le curve di loss per le top 10 configurazioni:



dove si può osservare un miglioramento sia in termini di velocità di convergenza che di valore finale all'aumentare del learning rate. Si ha la stessa sovrapposizione dei grafici vista in precedenza.

Curva ROC

Di seguito le curve ROC delle top 10 configurazioni:



Dove si nota che le curve ROC sono tutte equiparabili.

Stabilità Previsioni

La figura [Stability Comparison](#) mostra la stabilità delle previsioni, dove non si osserva un netto miglioramento al variare dei parametri delle configurazioni.

3.3.4 Scenario Regularizzazione

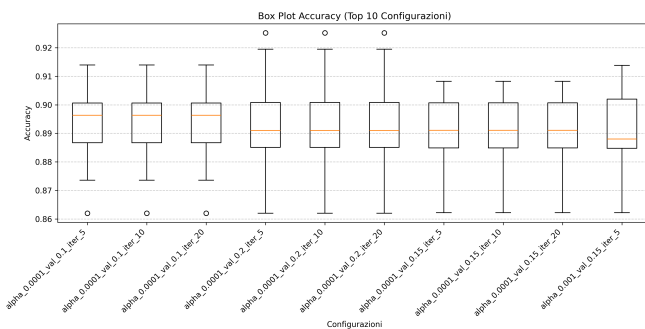
Box Plot

La figura [Box Plot All](#) mostra le box plot di tutte le configurazioni.

Si può notare:

- La variazione del numero di iterazioni permesse senza miglioramento non influenza l'accuracy
- L'aumento di alpha degrada le prestazioni del modello, visibile dalla presenza di outlier e code di varianza più basse

Prendendo le top 10 configurazioni si ha:

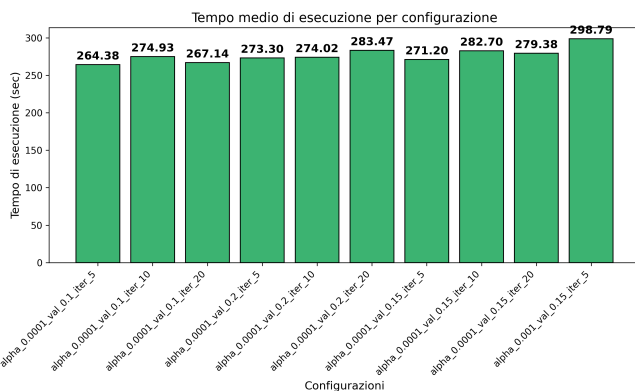


Tempi di Esecuzione

La figura [Execution Time All](#) mostra i tempi di esecuzione di tutte le configurazioni.

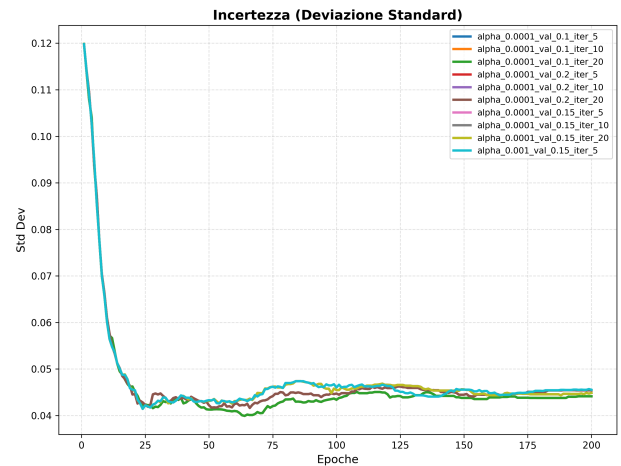
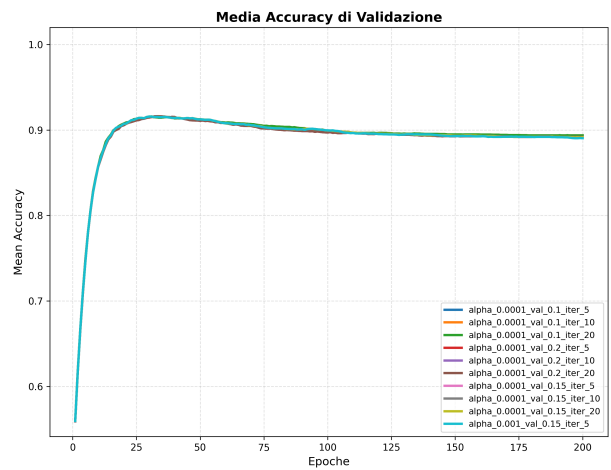
Questi sono tutti comparabili, segno che si ha una convergenza molto veloce.

Prendendo le top 10 configurazioni si ha:



Confronto Accuracy

La seguente immagine fa un confronto dell'accuracy media e della sua deviazione nelle top 10 configurazioni:

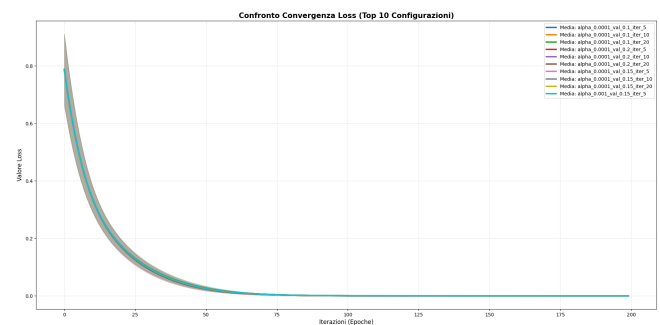


dove si può osservare:

- Accuracy media equirabile sulle configurazioni scelte
- Incertezza sull'accuracy presenta un leggero miglioramento al diminuire di alpha

Curva di Loss

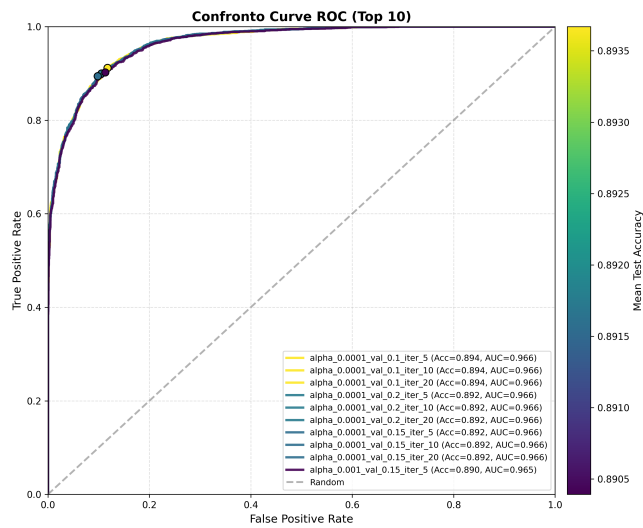
La prossima figura mostra le curve di loss per le top 10 configurazioni:



in cui si nota che tutte le configurazioni sono sovrapposte.

Curva ROC

Di seguito le curve ROC delle top 10 configurazioni:



dove si ha equivalenza tra le configurazioni.

Stabilità Previsioni

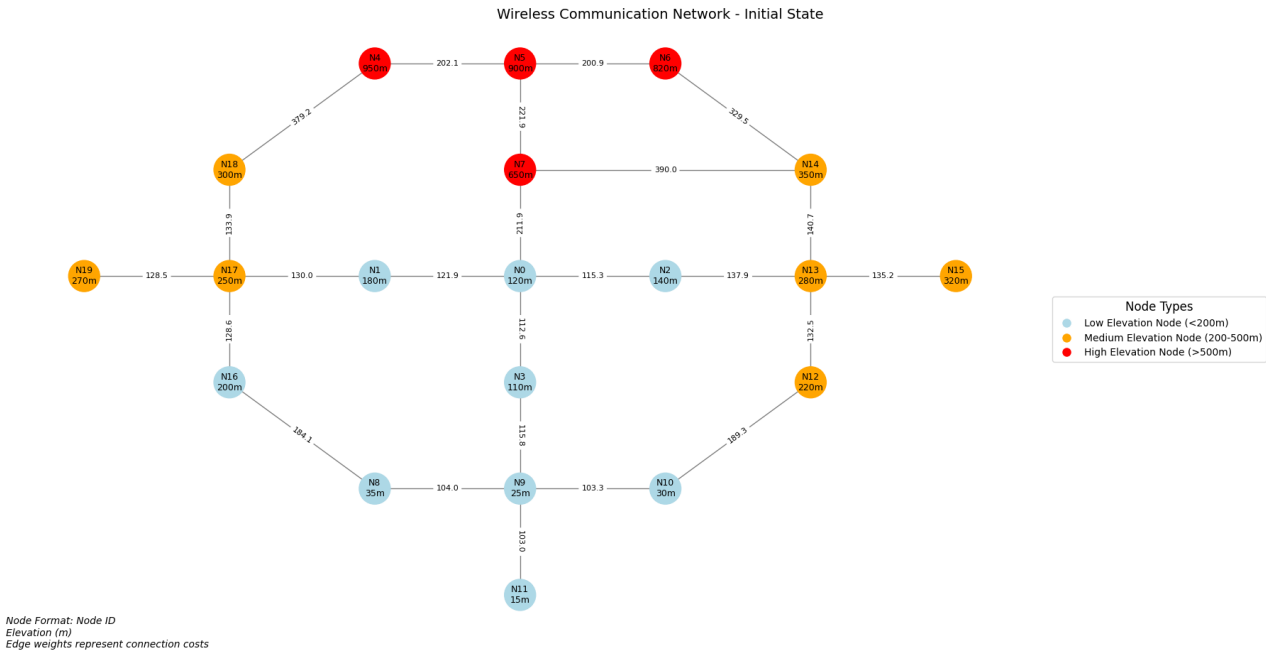
La figura [Stability Comparison](#) mostra la stabilità delle previsioni: più il valore è vicino a 1, più la classe predetta è stata 1.

Si può osservare come le configurazioni con alpha basso e validation frac = 0.1 siano leggermente migliori delle altre.

A Immagini

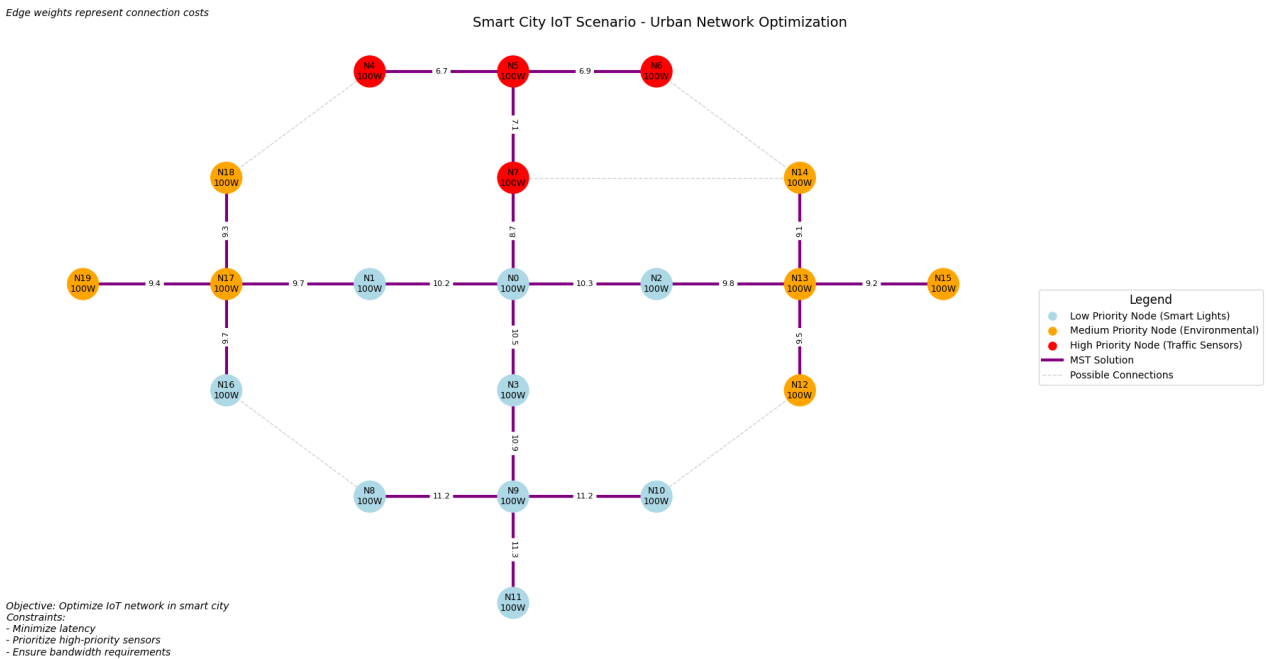
A.1 Tesina 1

Rete di default (report):



A.1.1 Smart City

Rete Smart City IoT (report):



Terminale Smart City IoT(report):

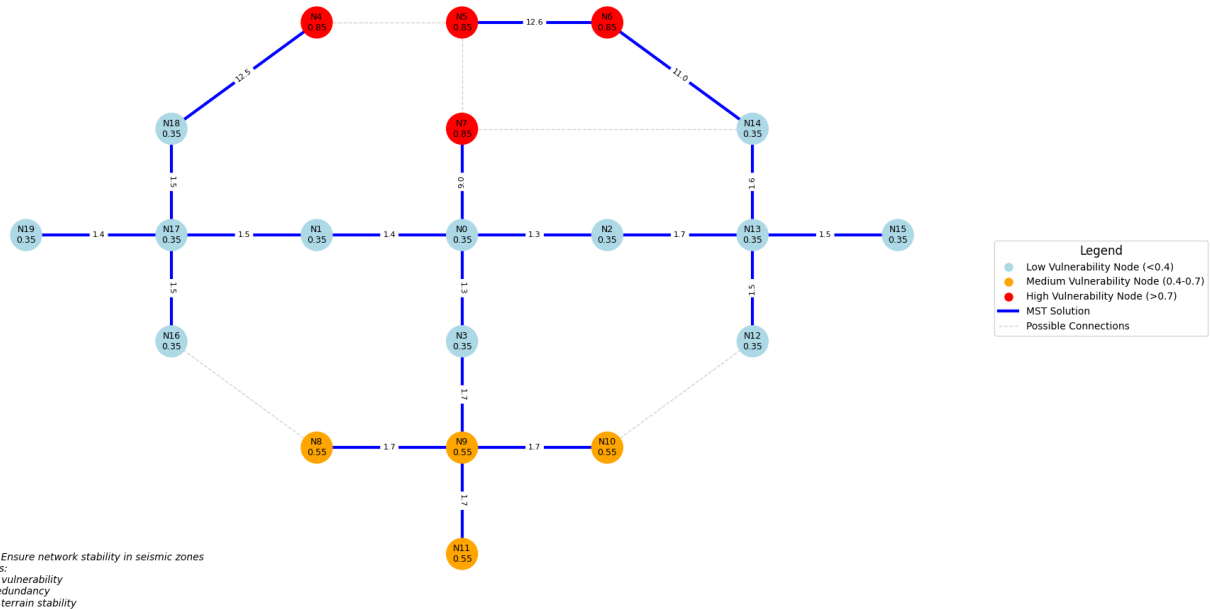
```
i Initializing Wireless Communication Network...
i Running smartcity scenario: Smart City IoT scenario - Optimizing connections for urban sensors
i Found optimal MST! Total cost: 180.79
```

A.1.2 Zona Sismica

Rete Zona Sismica (report):

Edge weights represent connection costs

Seismic Zone Scenario - Network Resilience



Terminale Zona Sismica (report):

```

i Initializing Wireless Communication Network...
i Running seismic scenario: Seismic zone scenario - Ensuring network resilience
i Found optimal MST! Total cost: 68.15
X ERROR: MST found but doesn't meet required constraints!
! WARNING: Max vulnerability per node : 0.65
! WARNING: Link: node 0 - node 7, Vulnerability score = 0.99
! WARNING: Link: node 6 - node 14, Vulnerability score = 1.09
! WARNING: Link: node 4 - node 18, Vulnerability score = 1.17
! WARNING: Link: node 5 - node 6, Vulnerability score = 1.61

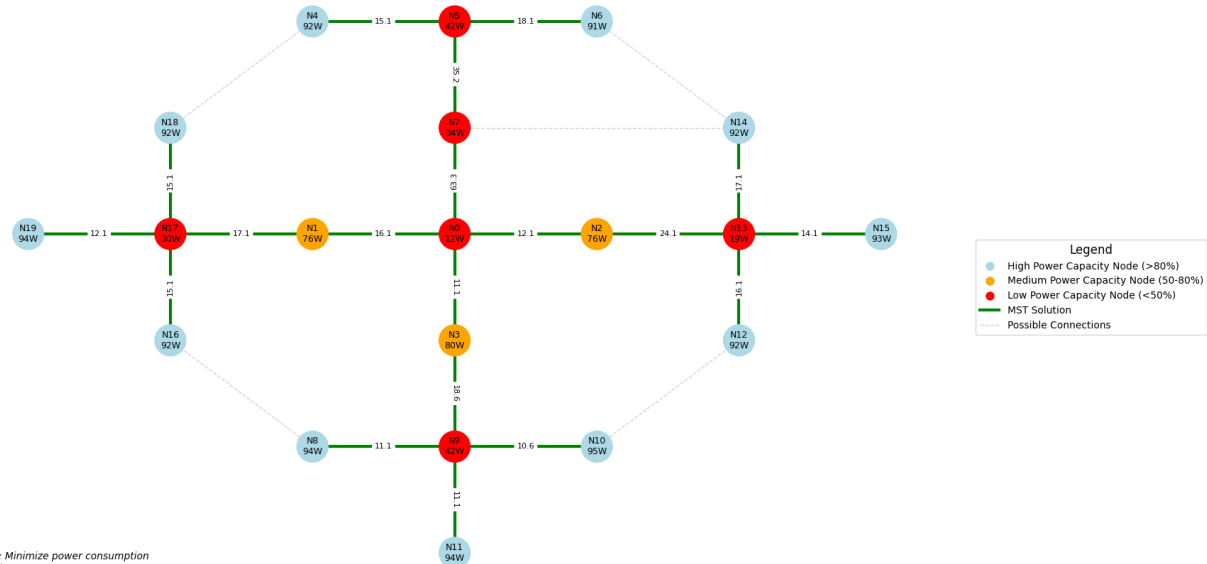
```

A.1.3 Ottimizzazione Energetica

Rete Ottimizzazione Energetica (report):

Edge weights represent connection costs

Energy Optimization Scenario



Objective: Minimize power consumption
 Constraints:
 - Stay within power budget
 - Balance load distribution
 - Optimize transmission paths

Terminale Ottimizzazione Energetica (report):

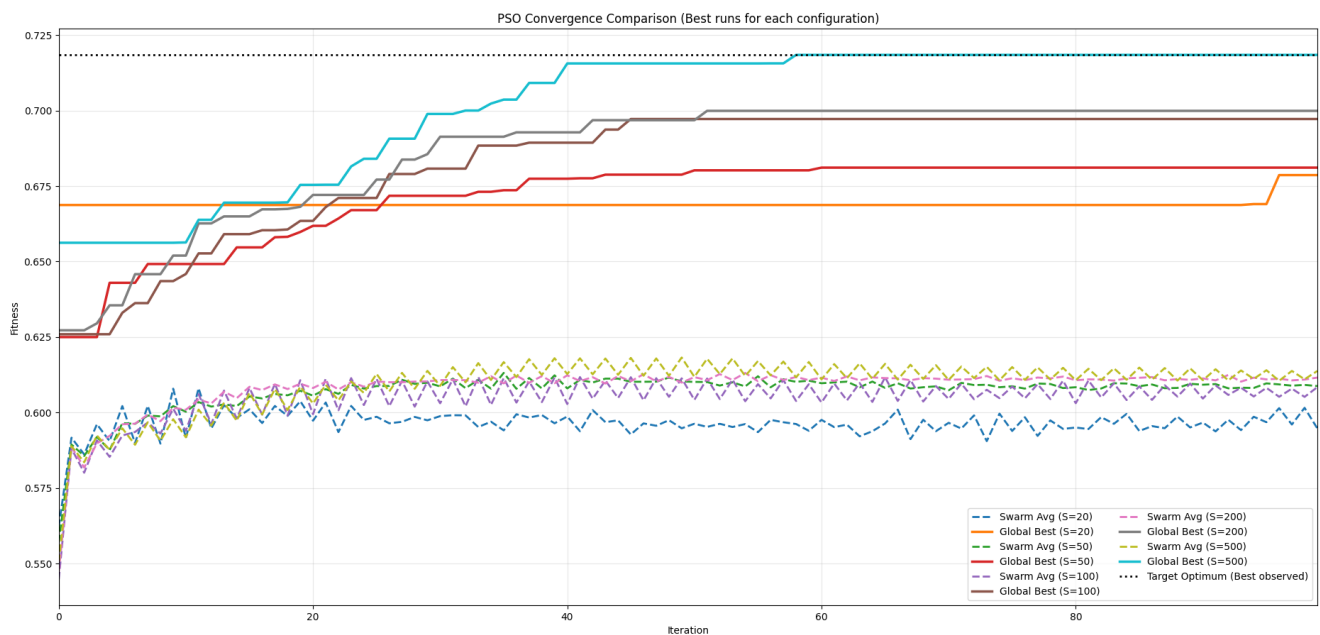
```
i Initializing Wireless Communication Network...
i Running energy scenario: Energy optimization scenario - Minimizing power consumption
i Found optimal MST! Total cost: 352.76
```

A.2 Tesina 2

A.2.1 Dimensioni Sciame

Convergenza

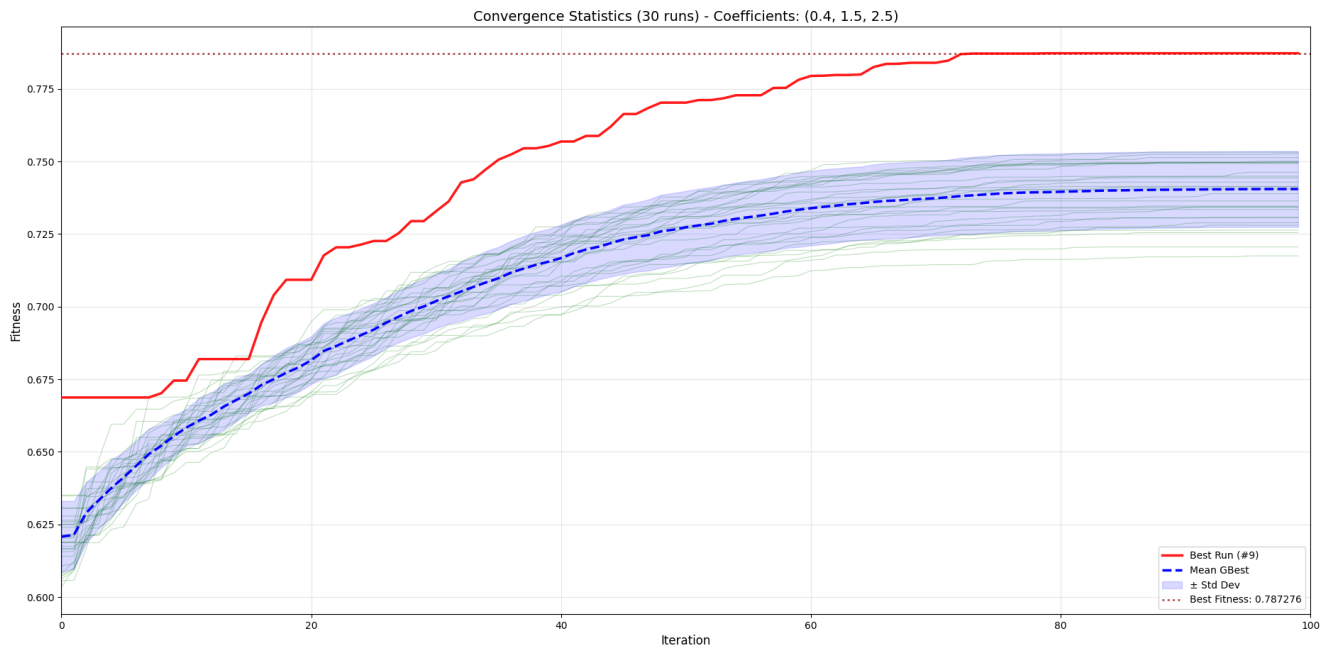
Convergenza di tutti gli sciame(report):



A.2.2 Coefficienti PSO

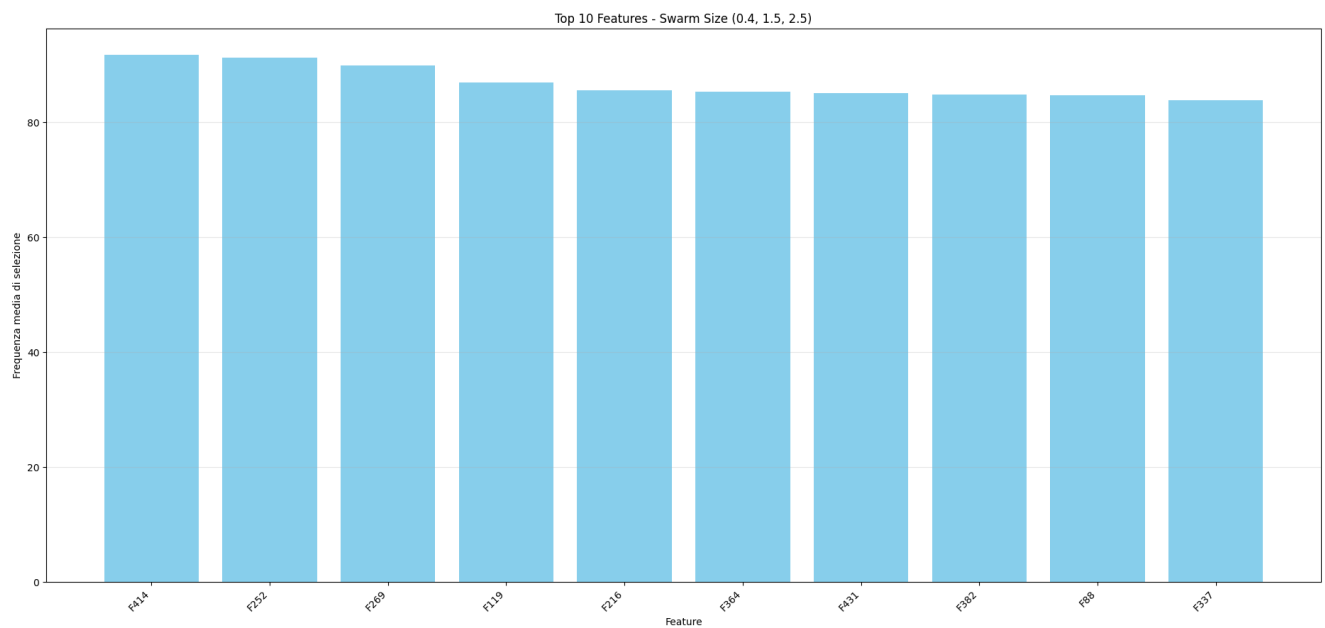
Convergenza

Convergenza della migliore configurazione ([report](#)):



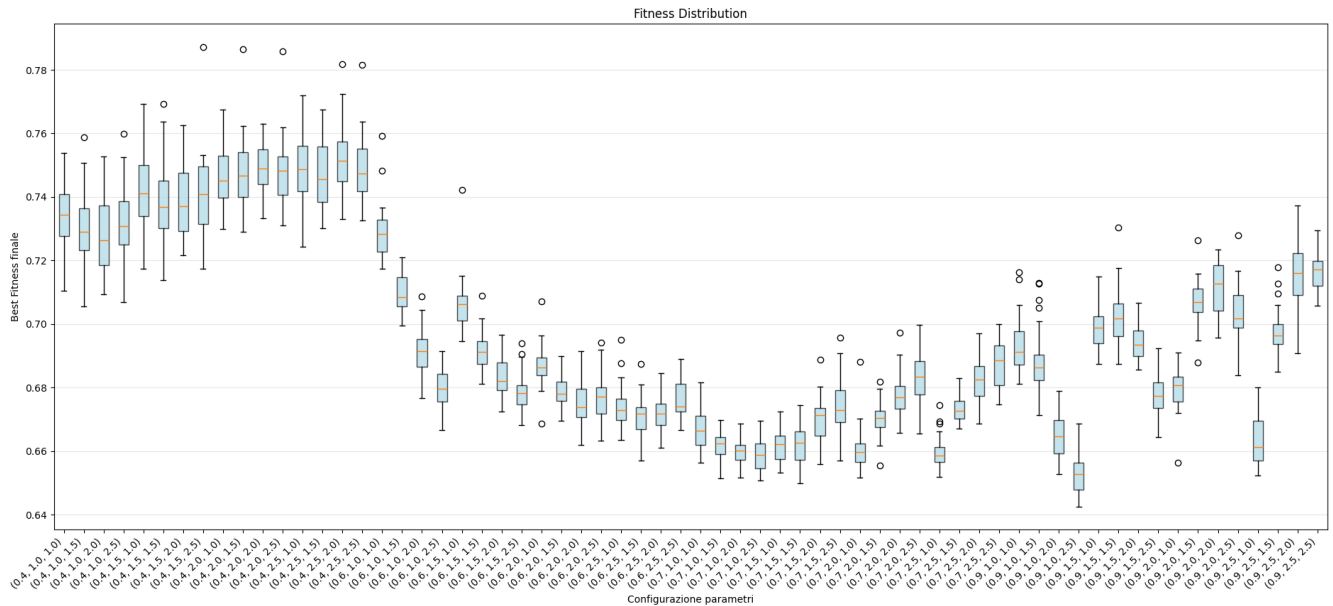
Feature

Feature selezionate dalla configurazione migliore ([report](#)):



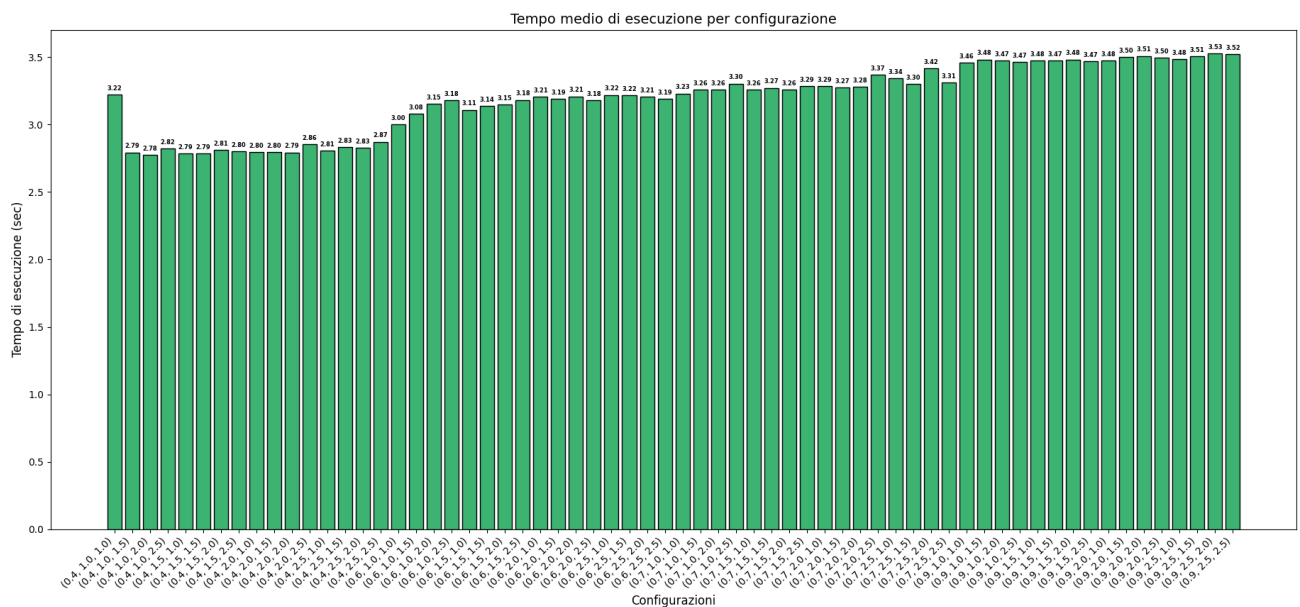
Box Plot

Box plot dell'accuracy di tutte le configurazioni ([report](#)):



Tempi di Esecuzione

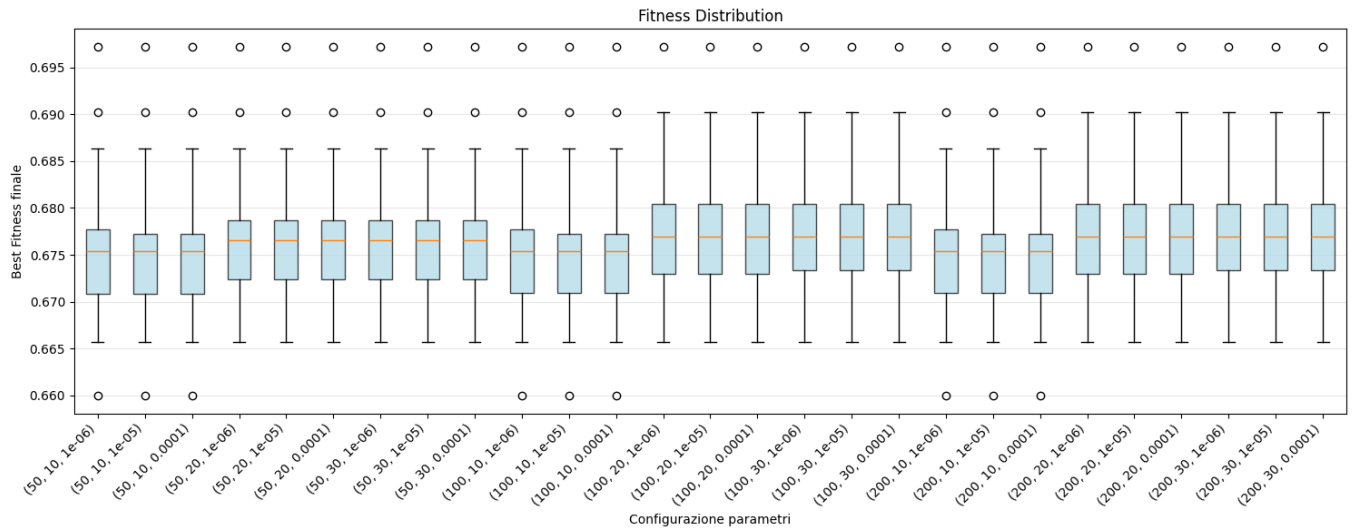
Tempi di esecuzione di ogni configurazione (report):



A.2.3 Criteri di Stop

Box Plot

Box plot dell'accuracy di tutte le configurazioni (report):

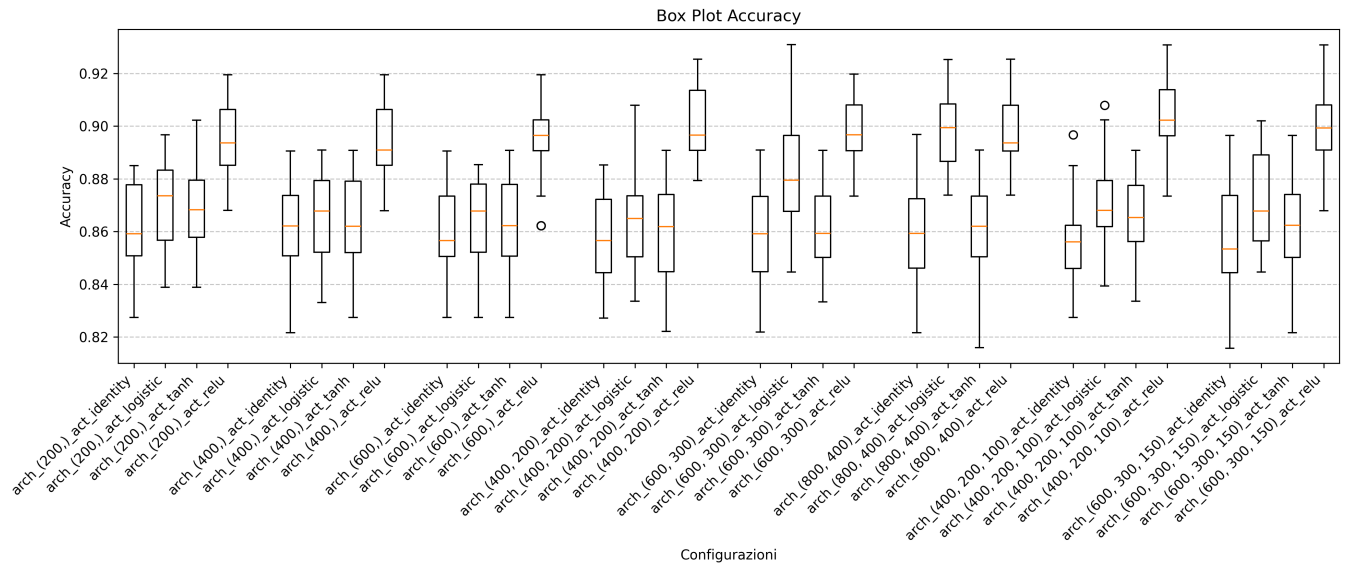


A.3 Tesina 3

A.3.1 Scenario - Architettura

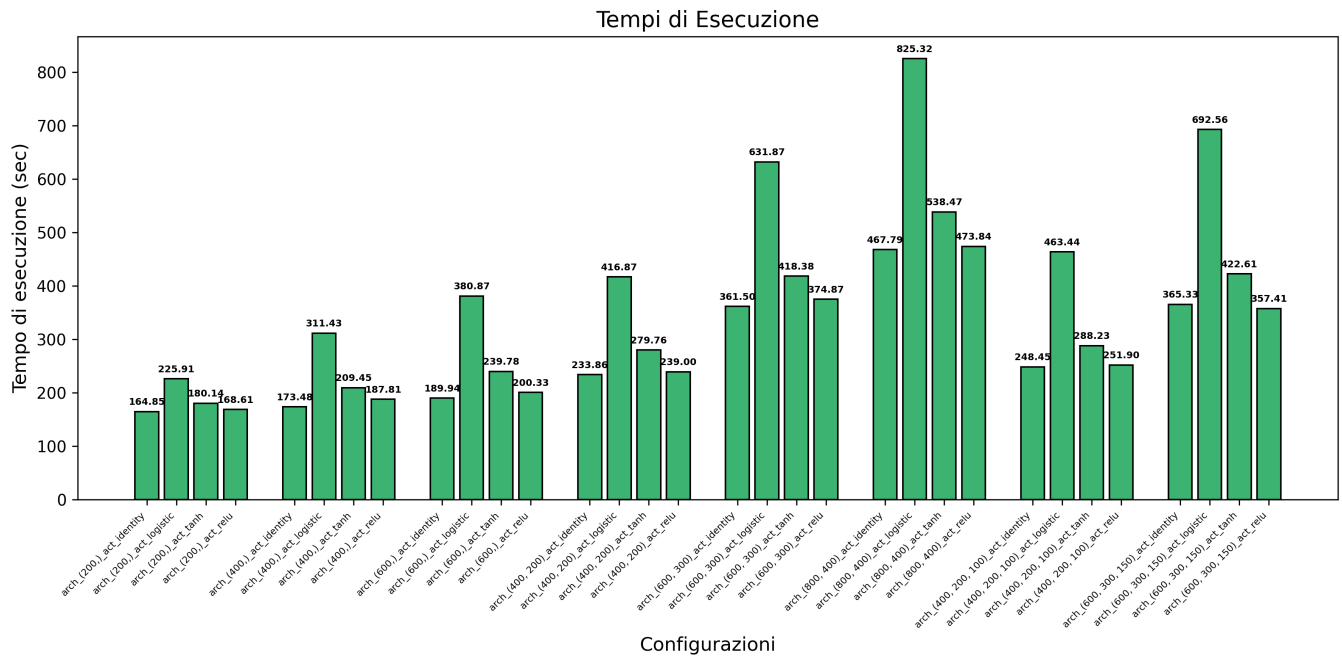
Box Plot

Box plot delle accuracy di tutte le configurazioni ([report](#)):



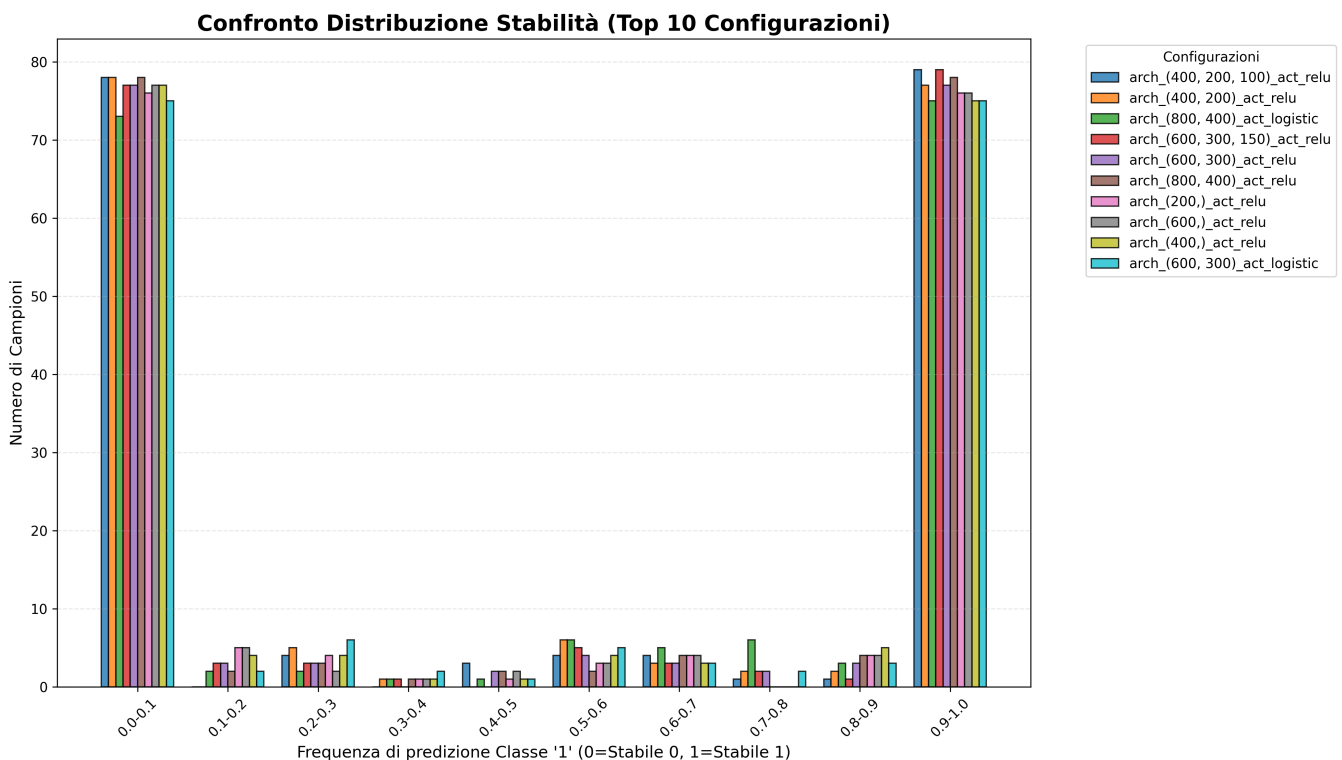
Tempi di Esecuzione

Tempi di esecuzione di tutte le configurazioni ([report](#)):



Stabilità Previsioni

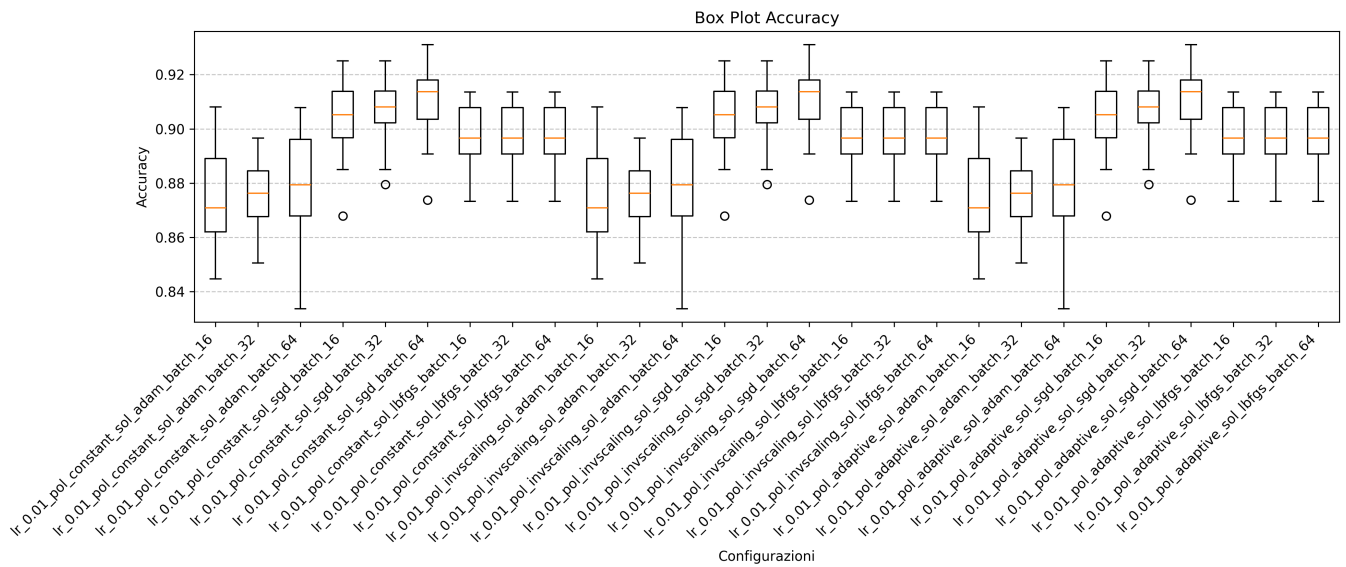
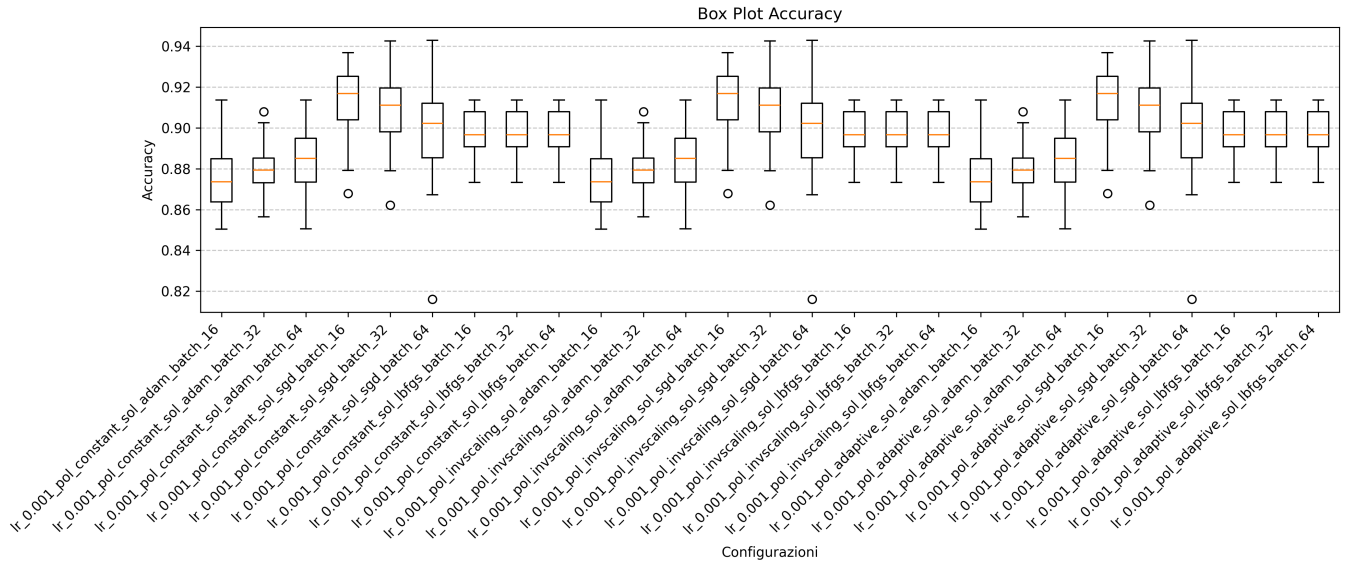
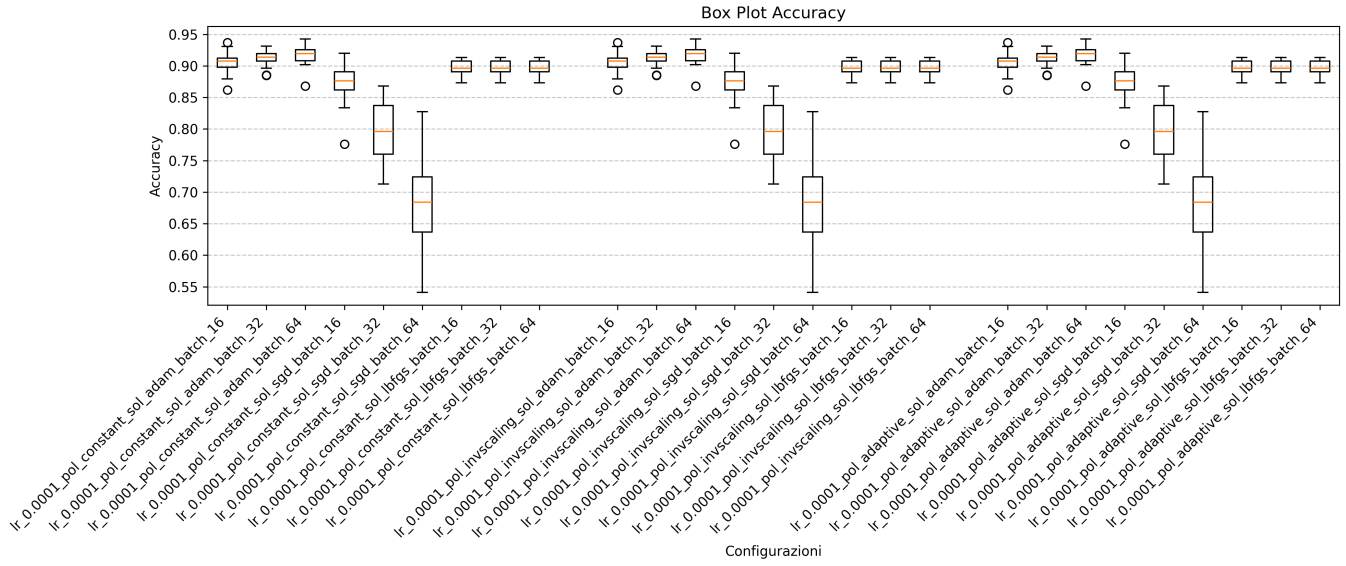
Stabilità delle previsioni delle top 10 configurazioni ([report](#)):

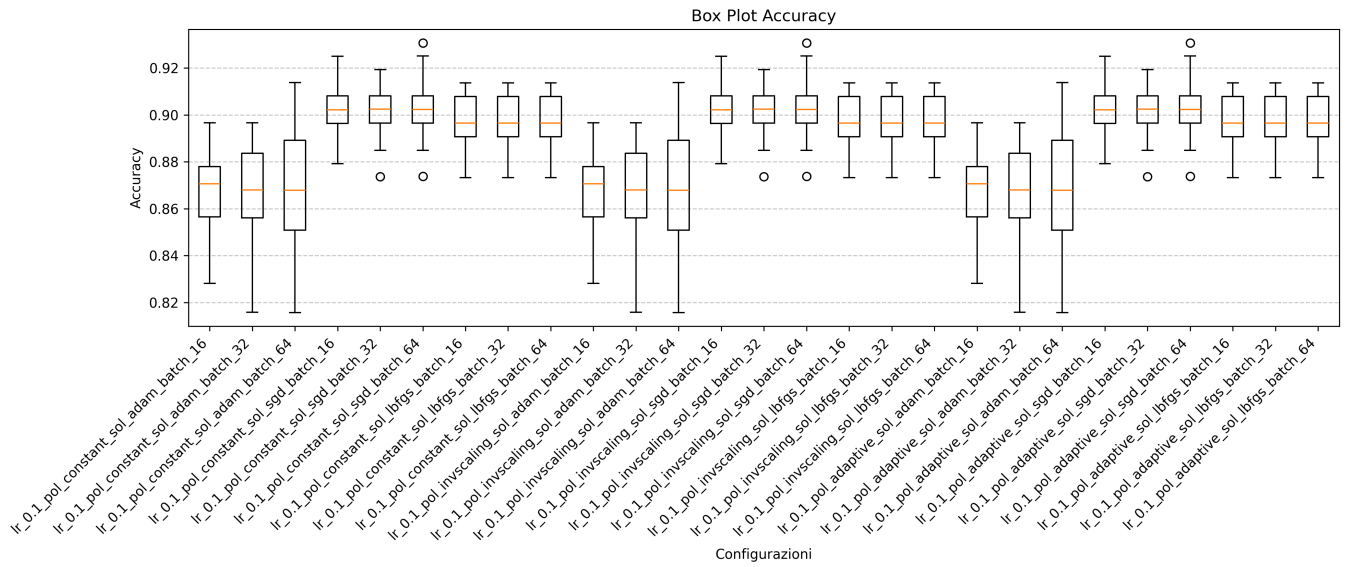


A.3.2 Scenario - Learning Rate

Box Plot

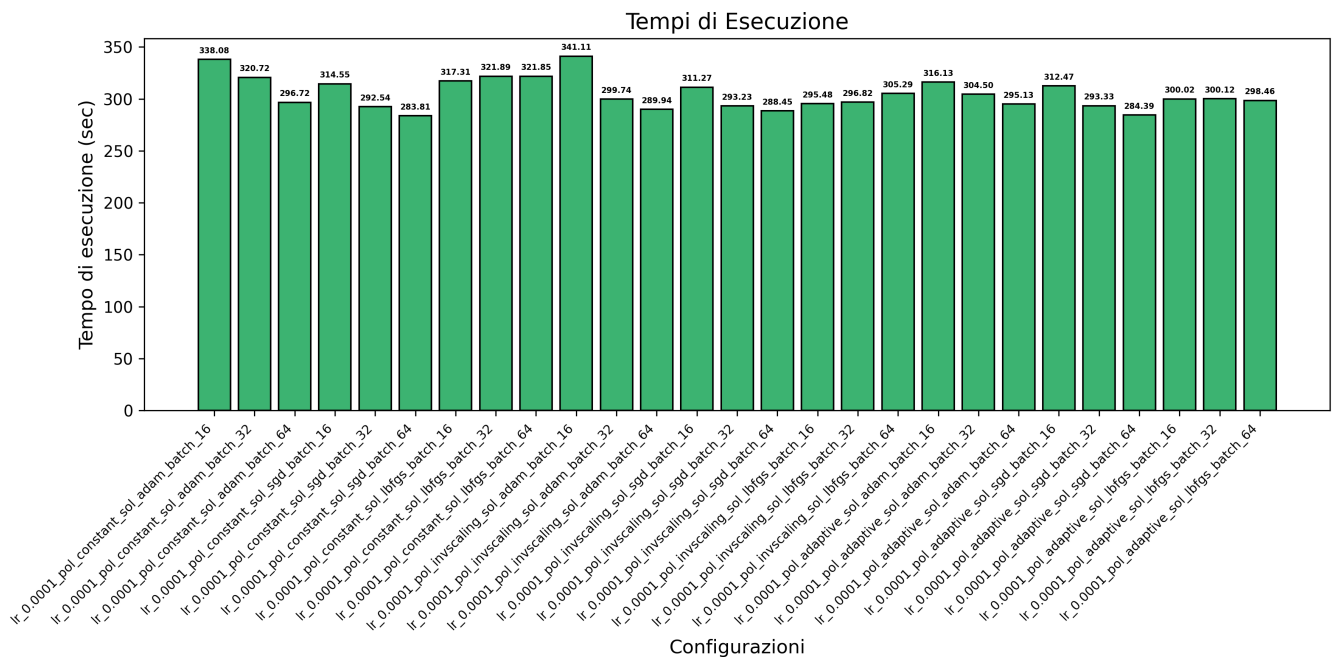
Box plot dell'accuracy di tutte le configurazioni ([report](#)):

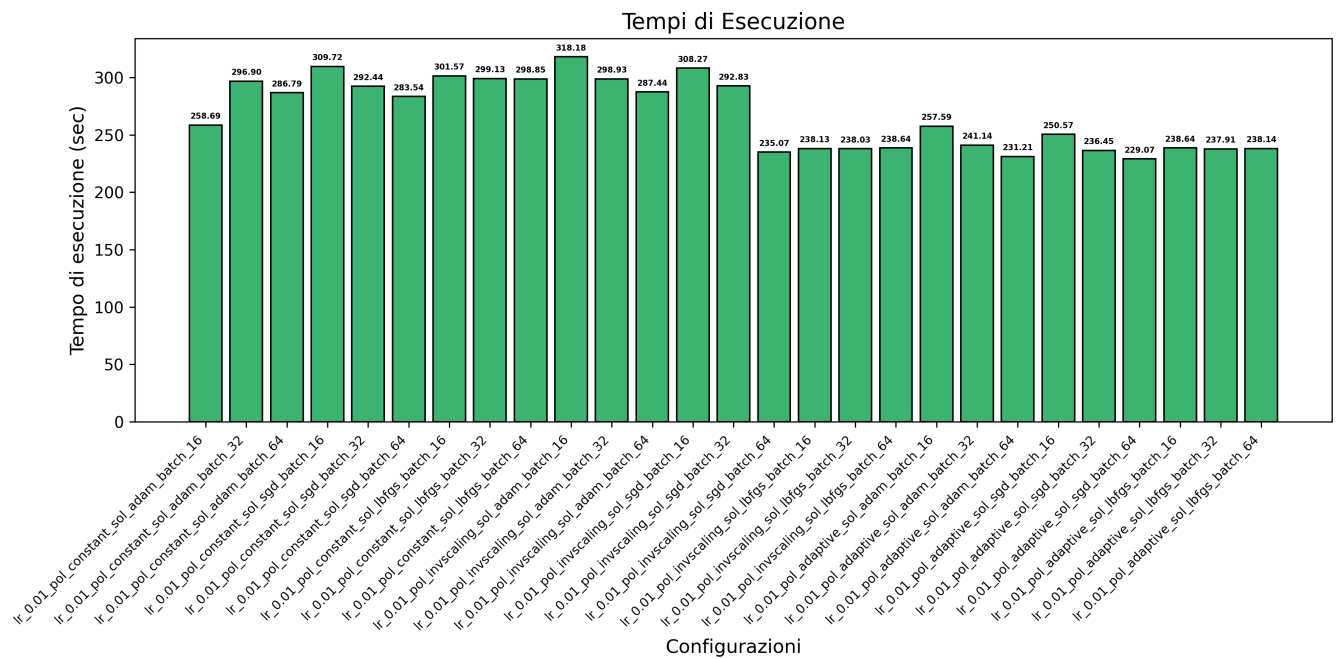
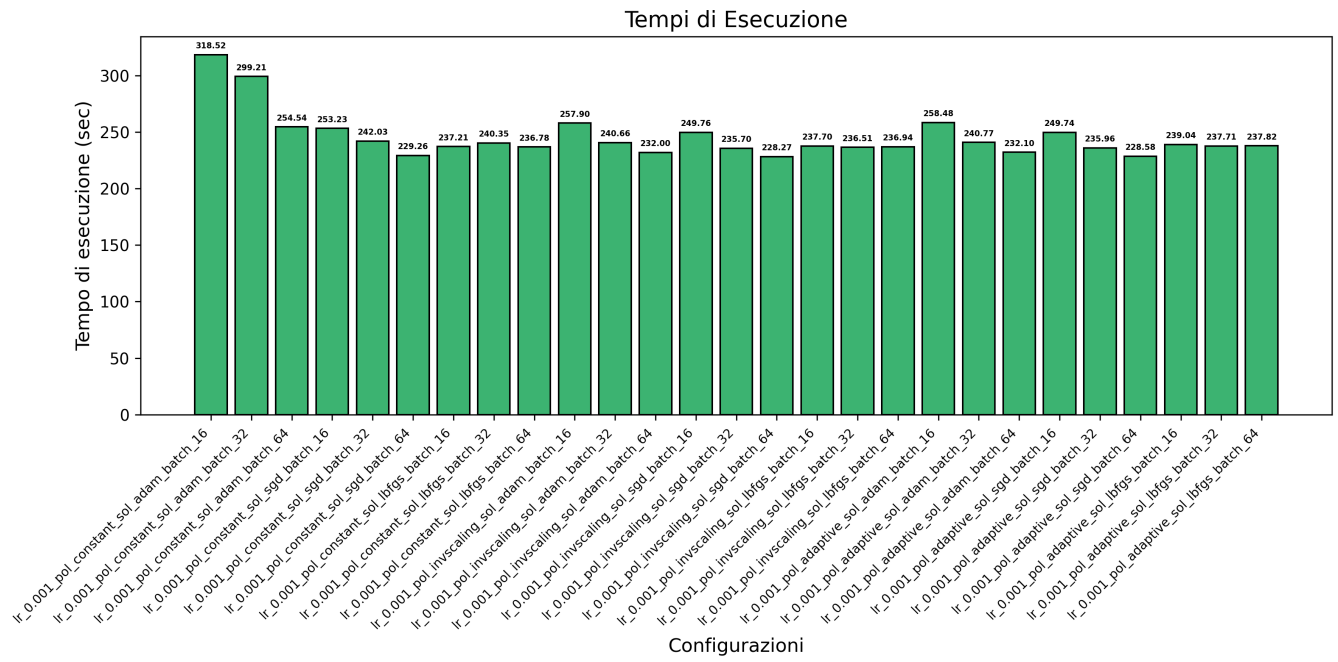


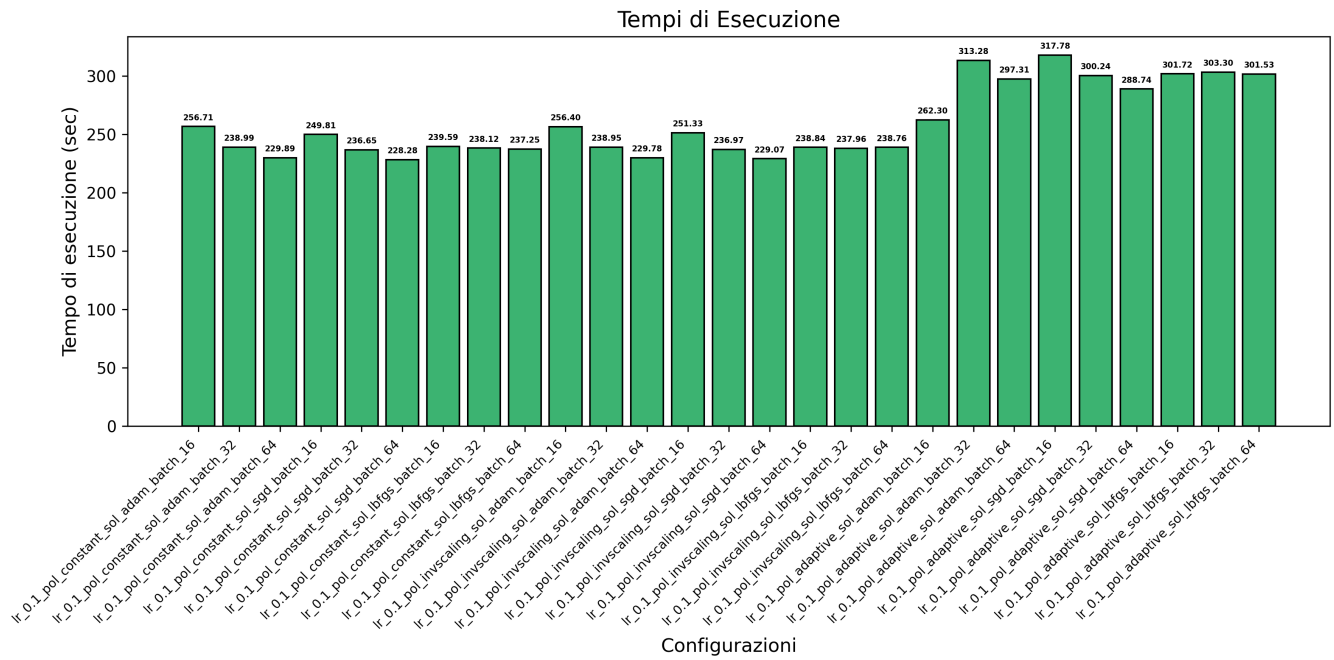


Tempi di Esecuzione

Tempi di esecuzione di tutte le configurazioni ([report](#)):

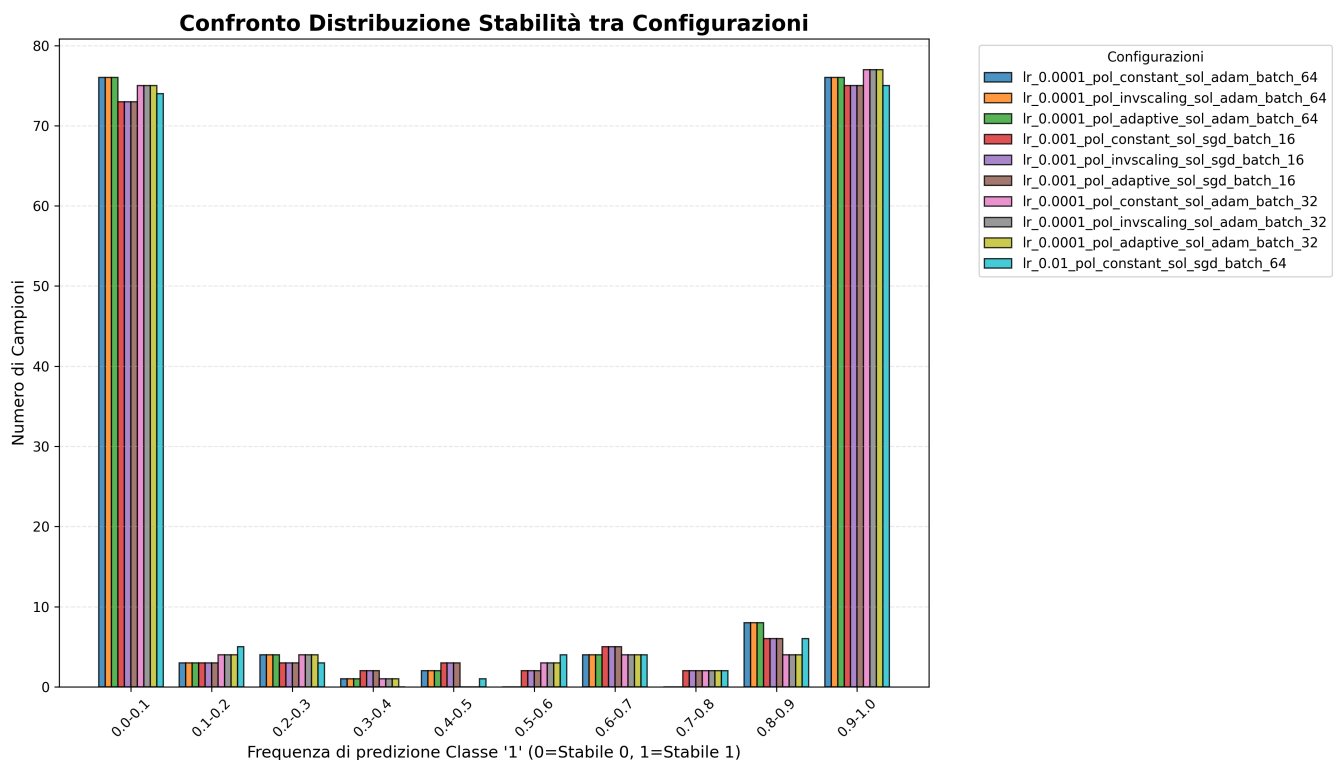






Stabilità Previsioni

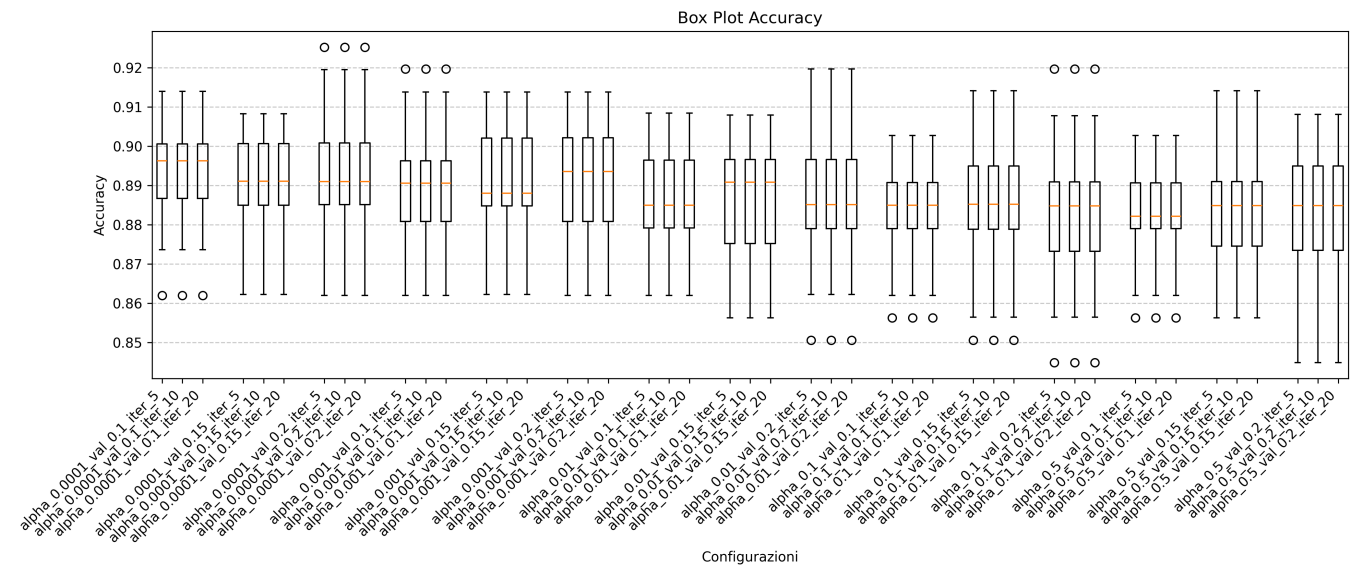
Stabilità delle previsioni delle top 10 configurazioni ([report](#)):



A.3.3 Scenario - Regolazione

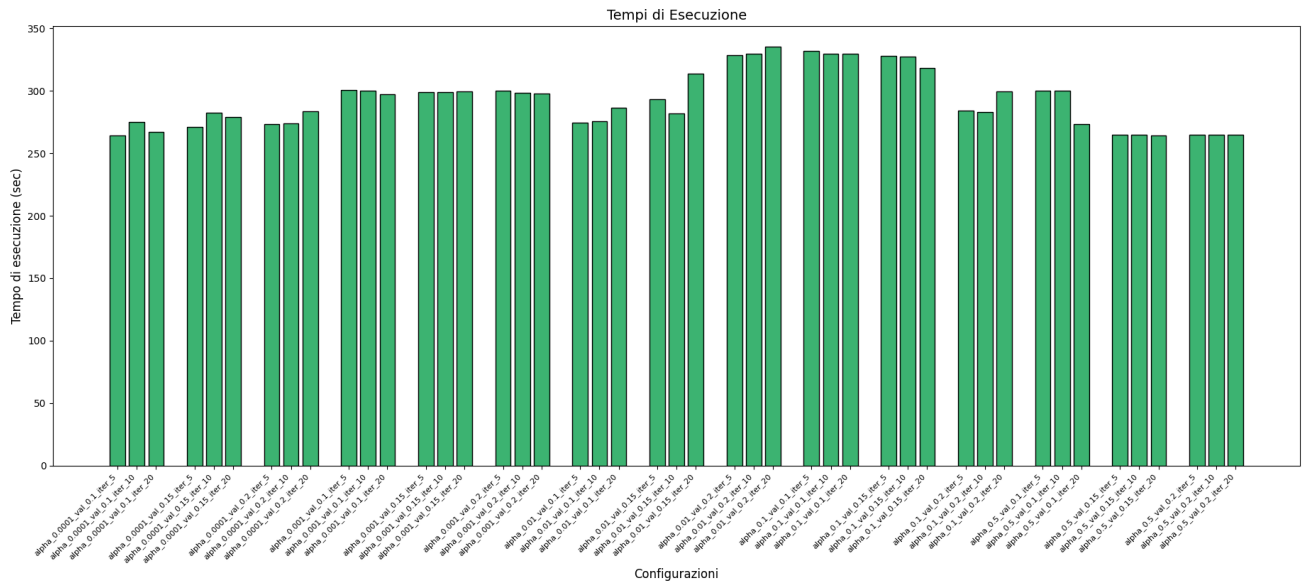
Box Plot

Box plot delle accuracy di tutte le configurazioni ([report](#)):



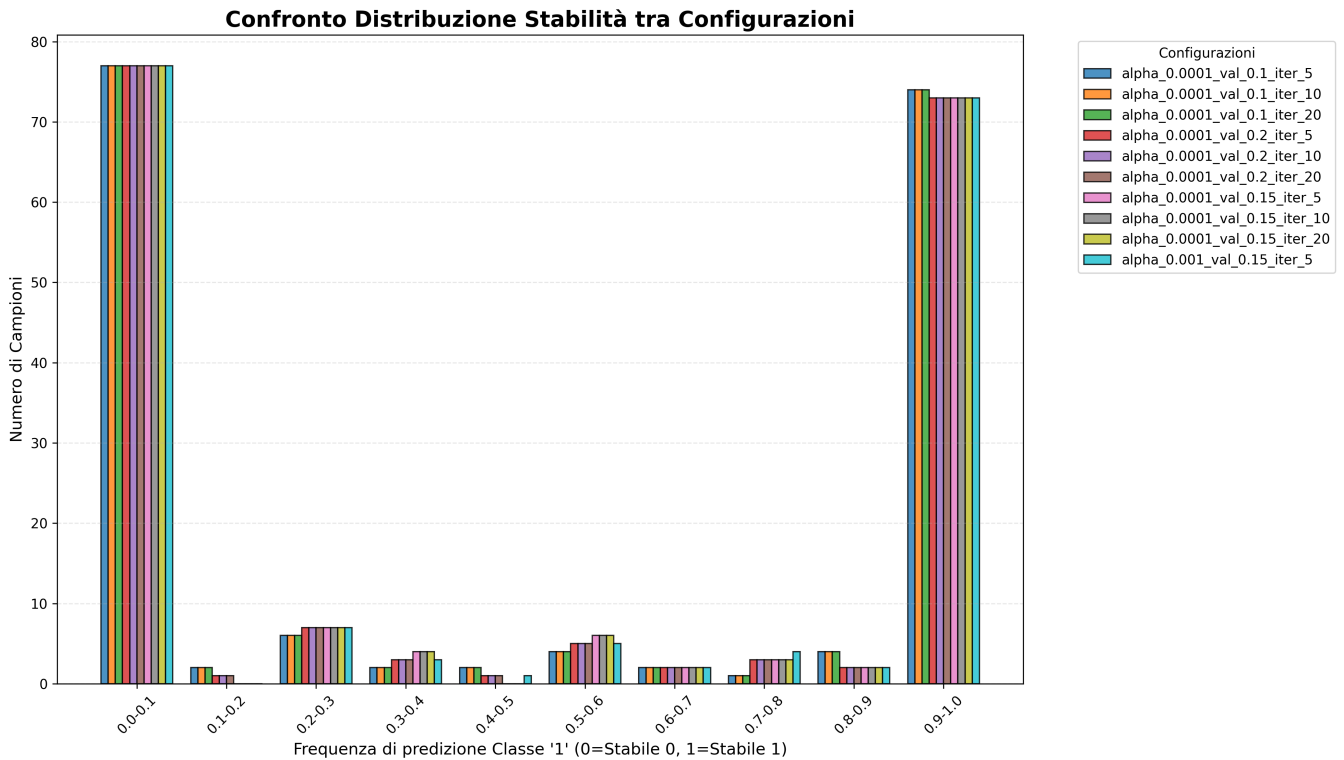
Tempi di Esecuzione

Tempi di esecuzione di tutte le configurazioni (report):



Stabilità Previsioni

Stabilità delle previsioni delle top 10 configurazioni (report):



B Codice

B.1 Tesina 1

Listing 1: Funzione di Costo Smart City (report)

```

1 def calculate_cost(self, node1: Node, node2: Node) -> float:
2
3     distance = node1.distance_to(node2)
4
5     # Latency factor (longer distance = higher latency = higher cost)
6     latency_cost = distance * 0.1 # 0.1ms per unit distance
7
8     # Penalizzo se sfiora il limite
9     if latency_cost > self.max_latency:
10         latency_cost = 3.0 * latency_cost
11
12     # Priority factor (lower priority = higher cost multiplier)
13     # Using terrain_difficulty as a proxy for priority (1.0 = high priority)
14     avg_priority = (node1.terrain_difficulty + node2.terrain_difficulty) / 2
15     priority_multiplier = 1/avg_priority
16
17     # Bandwidth efficiency
18     band_cost = avg_priority * self.bandwidth_factor
19
20     # Penalizzo se sfiora il limite di carica (visto in smartcity.py)
21     if band_cost > 3.0:
22         band_cost = 3.0 * band_cost
23
24     return (latency_cost + band_cost) * priority_multiplier

```

Listing 2: Funzione di Costo Zona Sismica (report)

```

1 def calculate_cost(self, node1: Node, node2: Node) -> float:
2
3     vuln_score = node1.get_vulnerability_score(node2)
4     # Penalizzo se sfiora il limite
5     if vuln_score > self.max_vulnerability:
6         vuln_score = 3.0 * vuln_score
7

```

```
8         # Metrica ideata per la stability
9         needed_power = node1.get_power_requirement(node2)
10        actual_power = (node1.power_capacity + node2.power_capacity) / 2
11        stability_factor = actual_power/needed_power
12
13        stability_penalty = 1/stability_factor
14
15        # Utilizzo del redundancy factor per accentuare il costo
16        return (vuln_score+stability_penalty)*self.redundancy_factor
```

Listing 3: Funzione di Costo Ottimizzazione Energetica (report)

```
1 def calculate_cost(self, node1: Node, node2: Node) -> float:
2
3     required_power = node1.get_power_requirement(node2)
4
5     power_per_node = required_power/2
6
7     power_cost = required_power
8
9     if power_per_node > self.max_power_per_node:
10         power_cost = 3.0*power_cost
11
12     node_load = power_per_node/node1.power_capacity
13
14     if node_load > 0.8: # Se la potenza richiesta supera l'80% della capacity del nodo
15         load_cost = node_load*3.0
16     else:
17         load_cost = node_load
18
19     return power_cost + load_cost
```